

Non-Negative Stiefel Approximating Flow: Orthogonalish Matrix Optimization for Interpretable Embeddings

Brian B. Avants, Nicholas J. Tustison, and James R. Stone
Department of Radiology and Medical Imaging
University of Virginia, Charlottesville, VA 22903

October 11, 2025

Abstract

Interpretable representation learning is a central challenge in modern machine learning, particularly in high-dimensional settings such as neuroimaging, genomics, and text analysis. Current methods often struggle to balance the competing demands of interpretability and model flexibility, limiting their effectiveness in extracting meaningful insights from complex data. We introduce Non-negative Stiefel Approximating Flow (NSA-Flow), a general-purpose matrix estimation framework that unifies ideas from sparse matrix factorization, orthogonalization, and constrained manifold learning. NSA-Flow enforces structured sparsity through a continuous balance between reconstruction fidelity and column-wise decorrelation, parameterized by a single tunable weight. The method operates as a smooth flow near the Stiefel manifold with proximal updates for non-negativity and adaptive gradient control, yielding representations that are simultaneously sparse, stable, and interpretable. Unlike classical regularization schemes, NSA-Flow provides an intuitive geometric mechanism for manipulating sparsity at the level of global structure while simplifying latent features. We demonstrate that the NSA-Flow objective can be optimized smoothly and integrates seamlessly with existing pipelines for dimensionality reduction while improving interpretability and generalization in both simulated and real biomedical data. Empirical validation on the Golub leukemia dataset and in Alzheimer’s disease demonstrate that the NSA-Flow constraints can maintain or improve performance over related methods with little additional methodological effort. NSA-Flow offers a scalable, general-purpose tool for interpretable ML, applicable across data science domains.

1 Introduction

Modern machine learning increasingly faces the challenge of extracting interpretable structure from high-dimensional, correlated data. In domains such as neuroscience, genomics, or natural language processing, data matrices often encode overlapping sources of variation: voxels representing distributed brain activity, genes co-expressed across pathways or words co-occurring across topics. These correlations hinder modeling, making it difficult to disentangle meaningful latent factors arising from complex phenomena such as gene expression profiles in bioinformatics (Golub et al. 1999), term-document frequencies in topic modeling (Blei, Ng, and Jordan 2003), multi-view biological measurements in integrative omics (Stražar and Žitnik 2016), and user-item interactions in recommender systems (Koren, Bell, and Volinsky 2009).

Classical dimensionality reduction techniques like principal component analysis (PCA) and its sparse variants seek low-rank approximations with interpretable bases (Zou, Hastie, and Tibshirani 2006), while sparse canonical correlation analysis (CCA) extends this to multi-view correlations in biological data (Witten, Tibshirani, and Hastie 2009). However, enforcing sparsity and decorrelation remains challenging: traditional methods may over-regularize or lack intuitive controls for partial constraints. Non-negative matrix factorization (NMF) (Daniel D. Lee and Seung 2001b) offers parts-based, additive representations aligned with domain constraints, but suffers from rotational ambiguity, yielding entangled factors (Ding et al. 2006).

Orthogonal variants improve sparsity and identifiability by aligning factors with disjoint structures, with applications in sparse PCA (Zou, Hastie, and Tibshirani 2006), sparse CCA (Witten, Tibshirani, and Hastie 2009), and interpretable neural networks (Henaff et al. 2011). Yet, strict orthogonality often sacrifices fidelity, especially in noisy or heterogeneous data and motivates the need for partial decorrelation models.

Soft orthogonalization methods address this by penalizing deviations from orthonormality. Examples include Disentangled Orthogonality Regularization (DOR), which separates Gram matrix components for convolutional kernels (Wu 2023); Group Orthogonalization Regularization (GOR), which applies intra-group penalties for vision tasks (Kurtz, Bar, and Giryes 2023); λ -Orthogonality Regularization, which introduces thresholded penalties for representation learning (Ricci et al. 2025); and simpler approaches like Spectral Restricted Isometry Property (SRIP) (Goeßmann 2020) and Frobenius norm penalties for neural stability (Guo 2019). However, these methods are typically embedded in neural training pipelines and do not enforce non-negativity, limiting their applicability to domains where measurement units should be preserved. Advanced ONMF variants, such as variational Bayesian approaches (Rahiche et al. 2022), unilateral factorization (Li, Zhang, and Zhang 2023), and deep autoencoder frameworks (Yang and Xu 2021), improve robustness but enforce strict orthogonality or require full decomposition, reducing flexibility for one-sided refinement.

In contrast to these soft regularization techniques, Riemannian optimization approaches have been explored primarily for enforcing strict orthogonality constraints on manifolds such as the Stiefel manifold, where the feasible set is equipped with a differential structure for gradient-based updates. For instance, Nonlinear Riemannian Conjugate Gradient (NRCG) methods optimize orthogonal NMF by projecting gradients onto the tangent space and using retractions like QR decomposition to maintain exact orthonormality, while handling non-negativity through coordinate descent on the complementary factor (Zhang et al. 2016). This ensures convergence to critical points with near-perfect orthogonality but incurs higher computational costs compared to soft penalties, and it typically requires full enforcement rather than flexible deviations. Hybrid methods like Feedback Gradient Descent (FGD) attempt to bridge this gap by approximating manifold dynamics in Euclidean space with feedback terms to achieve stable near-orthogonality efficiently, outperforming traditional Riemannian methods in DNN training speed while rivaling soft constraints in overhead (Bu and Chang 2022). However, adapting such Riemannian-inspired techniques to incorporate soft orthogonality penalties alongside non-negativity remains a less explored avenue.

To address these gaps, we propose **Non-negative Stiefel Approximating Flow (NSA-Flow)**, a variational optimization algorithm that approximates a target (X_0) with a non-negative matrix $(Y \in \mathbb{R}_{\geq 0}^{p \times k})$. NSA-Flow balances fidelity, column orthogonality, and non-negativity through a single tunable parameter $(w \in [0, 1])$ influencing proximity to the Stiefel manifold. The simple parameterization of these constraints means that NSA-Flow allows practitioners to directly encode the desired level of sparsity/decorrelation (which are closely related in this framework) in their embeddings, without the need for complex regularization schemes or full orthogonality constraints. As such, NSA-Flow employs global soft orthogonality constraints to promote disjoint support across columns and foster interpretable bases. Formulated to stay near the Stiefel manifold (Edelman, Arias, and Smith 1998), it departs from Riemannian gradient descent (Absil, Mahony, and Sepulchre 2008) in treating orthogonality as an objective rather than a constraint, so no retraction is required and no factorisation appears in the iteration. Non-negativity is the one hard constraint, enforced exactly by proximal projection (Parikh and Boyd 2014), and descent is guaranteed by a line search rather than by manifold geometry. Conceptually, NSA-Flow functions as a soft projection operator that can be inserted into any existing machine learning system to improve interpretability—whether as a regularization layer in a neural network (Henaff et al. 2011; Ricci et al. 2025), a refinement step in factor models (Li, Zhang, and Zhang 2023; Rahiche et al. 2022), or a sparsity-enforcing module in linear embeddings (Guo 2019; Goeßmann 2020). Unlike purely regularization-based methods (Wu 2023; Kurtz, Bar, and Giryes 2023; Ricci et al. 2025), NSA-Flow may also operate as a one-sided projection operator, preserving input structure while enabling controlled decorrelation.

Our contributions are:

1. A general framework for constrained matrix approximation, parameterized by (w) to intuitively control sparsity and orthogonality for interpretable ML.
2. Rigorous empirical validation that demonstrates good convergence properties and reliable benchmark

performance compared to baselines.

3. Broad applications, including enhanced disease classification on the Golub leukemia dataset, non-negative sparse PCA for biological integration and interpretable brain network discovery—showcasing NSA-Flow’s versatility across ML domains.
4. An [open-source implementation](#) in `pytorch` facilitating easy integration into existing workflows for researchers and practitioners. NSA-flow is also `pip` installable via the `nsa-flow` package and wrapped in R via `ANTsR`.

The paper is organized as follows: Section 2 derives the formulation and algorithm; Section 3 details the experimental results; Section 4 discusses limitations and future work; Section 5 gives an overview of software resources.

2 Methods

We consider the problem of finding a matrix $Y \in \mathbb{R}^{p \times k}$ that optimally approximates a target matrix $X_0 \in \mathbb{R}^{p \times k}$ while satisfying column orthogonality and, optionally, non-negativity. This general formulation is central to a wide range of problems in machine learning and signal processing, including orthogonal dictionary learning, Independent Component Analysis (ICA), and the orthogonal Procrustes problem ([Daniel D. Lee and Seung 2001a](#); [Hyvärinen and Oja 2000](#); [Schönmamm 1966](#)). The NSA-Flow optimization problem is defined by the minimization of a composite energy function $E(Y)$:

$$\min_{Y \in \mathbb{R}^{p \times k}, Y \geq 0} E(Y) = (1 - w) L_{fid}(Y, X_0) + w L_{orth}(Y),$$

where $w \in [0, 1]$ balances the fidelity loss L_{fid} against the orthogonality loss L_{orth} . Both losses are *dimensionless by construction*, so no internal re-weighting, warm-up calibration or data-dependent scaling constant is required and w is a genuine convex weight. The fidelity term is the relative squared Frobenius distance

$$L_{fid}(Y, X_0) = \frac{\|Y - X_0\|_F^2}{\|X_0\|_F^2},$$

which is 0 at $Y = X_0$ and exactly 1 at $Y = 0$.

For the orthogonality loss, write $G = Y^\top Y / \text{tr}(Y^\top Y)$ for the trace-normalised Gram matrix. We use one of two penalties, and the distinction is not cosmetic:

$$D(Y) = \left\| G - \frac{1}{k} I_k \right\|_F^2, \quad C(Y) = \frac{1}{k(k-1)} \sum_{i \neq j} \cos^2(y_i, y_j),$$

with $\cos(y_i, y_j) = \langle y_i, y_j \rangle / (\|y_i\| \|y_j\|)$. D is scale invariant, lies in $[0, 1 - 1/k]$, and vanishes exactly on the scaled Stiefel manifold $\mathbb{R}_{>0} \cdot St(p, k)$; we report $\tilde{D} = D / (1 - 1/k)$ so that it lies in $[0, 1]$ and equals 1 at rank one. It admits the spectral reading

$$D = k \text{Var}(\lambda_i(G)) = \frac{1}{\text{EffRank}(Y)} - \frac{1}{k},$$

so it is a participation-ratio deficit, and $D \geq 1/r - 1/k$ whenever $\text{rank}(Y) = r < k$: rank collapse is penalised rather than rewarded.

Crucially, D decomposes as

$$D = \underbrace{\sum_{i \neq j} G_{ij}^2}_{\text{decorrelation}} + \underbrace{\sum_i (G_{ii} - \frac{1}{k})^2}_{\text{norm balance}},$$

so $D = 0$ demands mutual orthogonality **and** equal column norms — that is orthoNORMality. C demands orthogonality alone, at any column norms. Which is appropriate depends on the application: when the

components genuinely differ in magnitude, D charges a penalty for the very structure the method exists to produce, and C does not. In the data-anchored formulation of Section @ref(dataanchored) the two coincide in practice, because the reconstruction term already forces the column norms to agree.

The Euclidean gradient of the $\tilde{D}$ form is available in closed form,

$$\nabla_Y E(Y) = (1 - w) \frac{2(Y - X_0)}{\|X_0\|_F^2} + \frac{w}{1 - 1/k} \cdot \frac{4}{t^2} \left[YS - \frac{\|S\|_F^2}{t} Y \right], \quad S = Y^\top Y, \quad t = \text{tr } S,$$

requiring no factorisation. It satisfies the Euler identity $\langle \nabla D, Y \rangle = 0$, since D is homogeneous of degree zero, so the orthogonality term cannot alter $\|Y\|_F$; and $\|\nabla D\|_F \leq 8/\|Y\|_F$. For C the gradient is tangential to each column’s sphere separately, so C cannot alter *any* individual column norm.

2.1 The Optimization Challenge and the NSA-Flow Approach

A plain Euclidean gradient step, $Y \leftarrow Y - \eta \nabla_Y E(Y)$, does not respect the constraints. The conventional remedy is Riemannian optimisation: project the gradient onto the tangent space of the Stiefel manifold and apply a **retraction** to return to it (Absil, Mahony, and Sepulchre 2008; Boumal 2023). Retractions are theoretically sound but require an SVD or an inverse matrix square root at every step, and enforcing exact feasibility can fight the fidelity term by abruptly correcting the iterate’s geometry.

NSA-Flow takes the opposite view: keep non-negativity as a hard constraint, which is cheap to enforce exactly, and treat orthogonality as an *objective* rather than a constraint. Nothing is retracted, so no factorisation appears in the loop. Figure 1 illustrates how the iterates evolve under different w .

2.1.1 Solver: spectral projected gradient

The feasible set $\{Y \geq 0\}$ is closed and convex and its Euclidean projection is the elementwise clamp $\Pi(Y) = \max(0, Y)$, so the problem is a smooth objective on a convex set and admits a projected-gradient method. We use the spectral projected gradient of Birgin, Martínez and Raydan (Birgin, Martínez, and Raydan 2000): Barzilai–Borwein step lengths (Barzilai and Borwein 1988)

$$t_n = \frac{\langle s_n, s_n \rangle}{\langle s_n, r_n \rangle}, \quad s_n = Y_n - Y_{n-1}, \quad r_n = \nabla E(Y_n) - \nabla E(Y_{n-1}),$$

safeguarded by an Armijo backtracking line search on the projected step

$$Y_{n+1} = \Pi(Y_n - t_n \nabla E(Y_n)), \quad E(Y_{n+1}) \leq E(Y_n) - \sigma \frac{\|Y_{n+1} - Y_n\|_F^2}{t_n}.$$

The line search makes the energy monotonically non-increasing by construction, and for a continuously differentiable E on a closed convex set the method converges to a stationary point.

2.1.2 Stationarity certificate

The quantity

$$\gamma_n = \frac{\|\Pi(Y_n - t_n \nabla E(Y_n)) - Y_n\|_F}{t_n}$$

is the norm of the projected-gradient mapping. It vanishes exactly at a stationary point of the constrained problem, and unlike a gradient norm it is valid *on* the boundary, which is where non-negative solutions live. We therefore report γ_n as a computable certificate, and the solver reports which stopping rule fired — the certificate threshold, line-search exhaustion, or the iteration cap — rather than asserting convergence. Conditioning degrades as $w \rightarrow 1$, where the problem approaches its combinatorial limit: convergence takes tens of iterations at $w = 0.5$, hundreds at $w = 0.9$, and thousands beyond $w = 0.99$.

A degeneracy is worth naming. At $w = 1$ the fidelity term vanishes and D is scale invariant, so the scale of the returned Y is unconstrained; every scaled Stiefel matrix is optimal, and nothing selects among clusterings either. The implementation warns at $w = 1$ and reports the observed scale drift.

Table 1: Computational Complexity of NSA-Flow and Related Methods

Method	Dominant Operation	Complexity ($\$p \setminus k\$)$	Orthogonality	Notes
Euclidean GD	Gradient Computation	($O(pk^2)$)	(Unstable)	Fails to enforce constraints.
Full Polar Retraction	Gradient + SVD / Polar	($O(pk^2)$)	(Exact)	Costly, non-smooth updates.
NSA-Flow (ours)	Gradient + Polar + Interp.	($O(pk^2)$)	(Controlled)	Smoother, faster, practical.
Cayley Transform	Linear Solve ($(p \setminus \times p)$)	($O(p^3)$) or ($O(pk^2)$) (w/ low rank)	(Exact)	Prohibitive for large (p).

2.1.3 Why the penalty must be normalised per column, not globally

The unnormalised penalty $\|Y^\top Y - I\|_F^2$ is sensitive to the norm of Y : scaling $Y \rightarrow cY$ scales it by c^4 , which makes the optimisation poorly conditioned and the step size hard to choose. An earlier version of this work therefore used a globally normalised off-diagonal defect,

$$L_{orth,inv}(Y) = \frac{\|Y^\top Y - \text{diag}(\text{diag}(Y^\top Y))\|_F^2}{\|Y\|_F^4},$$

described as measuring the cosines of the angles between columns. **It does not**, and the discrepancy has a concrete cost. Dividing by the global $\|Y\|_F^4$ rather than by $\|y_i\|^2 \|y_j\|^2$ leaves a quantity that is invariant to a global rescaling but not to rescaling the columns individually, so it is not a function of the angles alone. More seriously, its zero set is too large: a matrix with one non-zero column and $k - 1$ zero columns attains $L_{orth,inv} = 0$ exactly, because a zero column is orthogonal to everything. The functional is therefore *minimised by rank collapse*, and the same matrix is scored 0.2 by C and 0.8 by D .

The two penalties defined above fix this in different ways. C normalises each column, so it is a genuine function of the angles and is invariant to per-column rescaling, which is the real gauge freedom of a basis; its diagonal charges a column whose norm has collapsed, which doubles as a degeneracy penalty. D instead adds the norm-balance term, and its floor $D \geq 1/r - 1/k$ at rank r penalises collapse directly. Either is a correct measure of what the method is asked to produce; the off-diagonal defect alone is not, and it is not used here.

2.1.4 Relationship to Alternative Manifold Optimization Methods

- **Cayley Transform:** An alternative for preserving orthogonality is the **Cayley transform**, which defines an exact retraction-free update. For a skew-symmetric matrix $A = \text{grad}E(Y)Y^\top - Y(\text{grad}E(Y))^\top$, the update $Y^{(t+1)} = (I - \frac{\eta}{2}A)^{-1}(I + \frac{\eta}{2}A)Y^{(t)}$ exactly preserves orthogonality (Gao et al. 2019). However, it requires solving a $p \times p$ linear system, making it computationally prohibitive for large p .
- **Riemannian Optimization Frameworks:** Standard toolboxes like **Manopt** (Boumal et al. 2014) implement sophisticated algorithms like Riemannian trust-region and conjugate gradient methods. NSA-Flow instead avoids the manifold machinery altogether — orthogonality is an objective, not a constraint — which makes it a drop-in component for existing ML or deep-learning pipelines and lets the degree of constraint satisfaction be chosen rather than fixed.

2.1.5 Computational Complexity

For tall-skinny matrices ($p \gg k$) each iteration costs one Gram product and two $[p, k] \times [k, k]$ products, i.e. $O(pk^2)$, with no SVD, eigendecomposition or QR anywhere in the loop — the asymptotic cost is the same as a retraction method’s, but the constant is much smaller and the per-step work is a few matrix multiplications rather than a factorisation. When the basis is fitted directly to data with $p > n$ the Gram matrix is never formed either: every quantity follows from XV and $X^\top(XV)$, giving $O(npk)$ per iteration and removing an allocation that is 0.4 GB at $p = 7129$ and 3.2 GB at $p = 20000$. Practically speaking, as will be shown in the Results section, NSA-Flow provides an intuitive “knob” that allows users to impose orthogonality, non-negativity and sparseness via a consistent and intuitive framework. Table 1 summarizes these comparisons.

Optimization trajectories diverging from manifold $\mathcal{M}$

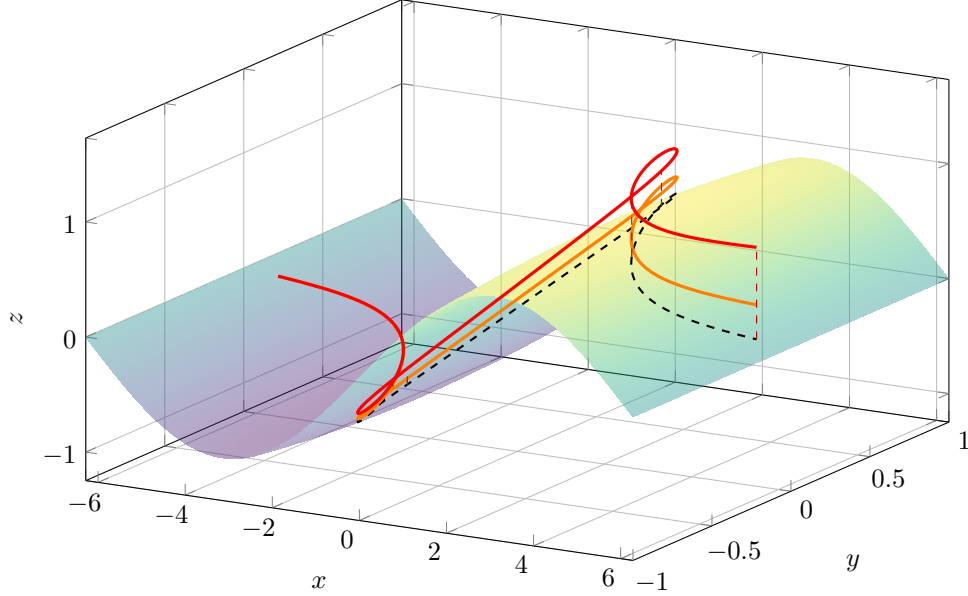


Figure 1: Illustration of the NSA-Flow optimization as a function of ω . The colored manifold is a conceptual representation of the Stiefel manifold, with the curves representing optimization paths for evolving Y . When ω is small the orthogonality term is weakly weighted and the iterate is free to deviate from orthonormality; when ω is large the term dominates and the solution is drawn toward the manifold.

2.1.6 Implementation

The NSA-Flow algorithm is implemented in `pytorch` as a small, dependency-light package; the results in this document are produced by calling it directly. The library is five modules and under a thousand lines, a consolidation from the earlier version’s eight modules and roughly 2400 lines, achieved by deleting rather than adding.

The refinement entry point accepts a target X_0 , an optional initialisation, and the weight w ; the estimation entry point accepts the data matrix X and the number of components k . What it no longer accepts is the earlier version’s configuration surface: there is no retraction type, no optimizer backend, no learning-rate strategy, no aggression or warm-up parameter, and no separate fidelity and orthogonality type switches. Those existed to compensate for an energy whose two terms were not comparably scaled, and they became unnecessary once both terms were made dimensionless. The document records exactly which arguments were dropped in Table @ref(tab:dropped).

Each iteration computes a fused value-and-gradient sharing a single Gram product, takes a Barzilai–Borwein step, and accepts it through an Armijo line search on the projected step; the energy is therefore monotone by construction. Non-negativity is applied by exact projection. Diagnostics recorded per iteration are the energy, the two loss terms, the step length and the projected-gradient mapping norm, and the returned object carries the stopping reason, the certificate value, the iteration count, the effective rank and the scale ratio.

Two numerical choices are worth stating because both were errors in the earlier version. The orthogonality defect is computed as $\|G - I/k\|_F^2$ directly rather than as $\|G\|_F^2 - 1/k$; the latter is algebraically identical but suffers catastrophic cancellation near the optimum and returns small negative values. And when a polar factor must be differentiated, the derivative is obtained by solving the Sylvester equation, whose denominators are $h_i + h_j > 0$, rather than through the SVD backward pass, whose denominators are $\sigma_i^2 - \sigma_j^2$ and which returns NaN for *every* entry when singular values repeat — as they do for any orthogonal initialisation.

The implementation is research-grade but tested: the accompanying suite asserts the propositions of this section executably, including the zero set, the bounds, the spectral identity, the rank-collapse floor, the invariances, the disjoint-support equivalence, agreement of every closed-form gradient with automatic differentiation, and equality of the Gram and matrix-free code paths. Experimentalists should still centre and scale their inputs and choose w for the application; unlike the earlier version, there is nothing else to tune.

2.2 Sparse PCA via NSA-Flow

Sparse principal component analysis (SPCA) finds a sparse basis $Y \in \mathbb{R}^{p \times k}$ that maximizes the variance explained in a data matrix $X \in \mathbb{R}^{n \times p}$. We can formulate SPCA as a regularized optimization over Y :

$$\min_Y \left\{ -\frac{1}{2n} \text{tr}(Y^\top X_c^\top X_c Y) + \lambda R(Y) \right\},$$

where X_c is the data matrix, $\lambda \geq 0$ is a regularization parameter controlling the regularization penalty, and $R(Y)$ is a suitable regularization function. The smooth component of the objective is the negative explained variance, $f(Y) = -\frac{1}{2n} \text{tr}(Y^\top S Y)$, where $S = X_c^\top X_c$ is the covariance matrix. The Euclidean gradient of this term is:

$$\nabla_Y f(Y) = -\frac{1}{n} S Y.$$

We optimize this energy in two main steps:

Step 1: Gradient Descent with Line Search A candidate update $Z^{(t)}$ is computed by taking a step along the negative gradient direction from the current iterate $Y^{(t)}$:

$$Z^{(t)} = Y^{(t)} - \alpha^{(t)} \nabla_Y f(Y^{(t)}),$$

where the step size $\alpha^{(t)}$ is determined by an Armijo-type backtracking line search to ensure sufficient decrease in the objective function.

Step 2: Proximal Step and Orthogonality Enforcement The non-smooth regularization term arising from R is handled in a proximal step. The method supports two types of proximal updates:

- **proximal_type = "basic" (Proximal Thresholding):** A standard soft-thresholding operator is applied to the candidate $Z^{(t)}$ to induce sparsity, followed by an optional non-negativity projection:

$$Y^{(t+1)} = \text{prox}_{\alpha\lambda, \|\cdot\|_1}(Z^{(t)}) = \text{sign}(Z^{(t)}) \odot \max(|Z^{(t)}| - \alpha^{(t)}\lambda, 0).$$

This standard approach decouples the sparsity and orthogonality steps. We re-orthogonalize via QR at the beginning of each iteration when this algorithmic path is taken.

- **proximal_type = "nsa_flow" (Proximal Flow):** A more sophisticated proximal step is performed by invoking the NSA-Flow algorithm. The candidate matrix $Z^{(t)}$ serves as the target for an inner NSA-Flow optimization loop:

$$Y^{(t+1)} = \arg \min_{U \geq 0} \left\{ \frac{1}{2} \|U - Z^{(t)}\|_F^2 (1 - w) + \text{Orth}(U)w \right\}.$$

This subproblem simultaneously encourages fidelity to the gradient-updated iterate $Z^{(t)}$, promotes sparsity through non-negativity, and enforces column orthogonality via the flow. This approach integrates the constraints more tightly into the optimization, providing a unified update that respects both the geometry and the regularization.

The algorithm terminates when the relative change in energy, the norm of the gradient, and the change in the iterate all fall below a predefined tolerance τ . An adaptive learning rate scheduler is also employed, which reduces the step size α if the objective function fails to improve for a set number of iterations (patience), thereby enhancing stability and preventing premature termination at plateaus. The final output is the set of sparse loadings Y that achieved the lowest energy during the optimization. This is one example

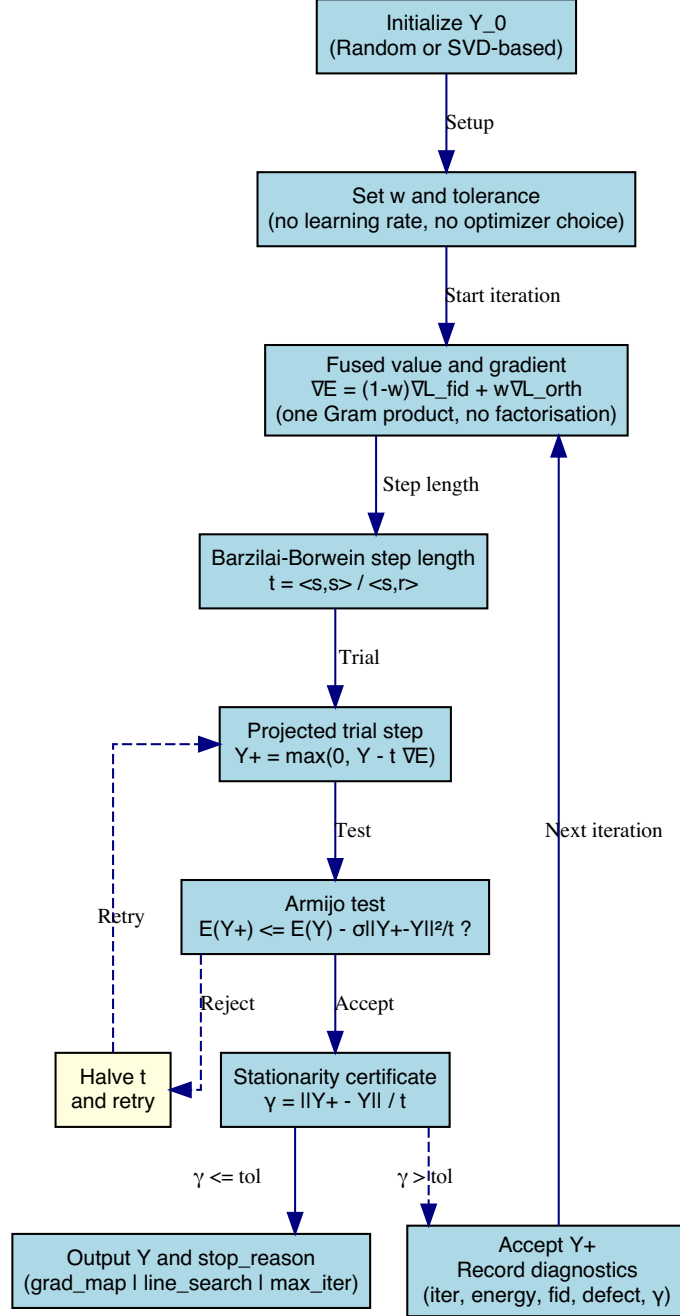


Figure 2: NSA-Flow Algorithm Workflow

where NSA-Flow can be integrated as a proximal operator within a broader optimization framework to enforce orthogonality and non-negativity. Other approaches (shown below) more directly use NSA-Flow as a standalone method for matrix approximation; for example, by directly approximating the loading matrix of PCA with the bases derived from NSA-Flow. Both SPCA and the latter approach are demonstrated in the Results section. The implementation is available in the `nsa_flow_pca` function within the ANTsR package. We implement a closely related approach for NSA-flow constrained factor analysis in the `nsa_flow_pca_fa` function which is illustrated in an accompanying documentation article (ANTsR).

3 Results

We use default settings in the results below with $w = 0.5$ unless otherwise specified. Because both energy terms are dimensionless, there is no relative weighting to set: w is the only balance parameter, and it is not a nuisance parameter that has to be calibrated per problem. There is likewise no learning rate to choose — Barzilai–Borwein steps are scale-free and the Armijo line search accepts or halves them — and convergence is declared on the projected-gradient mapping norm falling below 10^{-9} in double precision (10^{-6} in single, which is above float32 machine epsilon), or reported as unconverged otherwise. Empirically $w = 0.5$ is a better default than more aggressive settings: across the experiments below it dominates $w = 0.9$ on both reconstruction and reproducibility, because large w pins the solution near a combinatorial vertex where small perturbations of the data change which vertex is selected.

3.1 Toy Example: Decomposing a Small Mixed-Signal Matrix

To intuitively illustrate NSA-Flow, consider a toy 4x3 matrix X_0 representing mixed signals: each column is a nonnegative orthogonal basis vector (e.g., distinct patterns), but observed with noise and scaling. NSA-Flow approximates an orthogonal nonnegative basis Y close to X_0 . Figure 3 shows the results of applying NSA-Flow to this toy example, starting from a random initialization Y_0 .

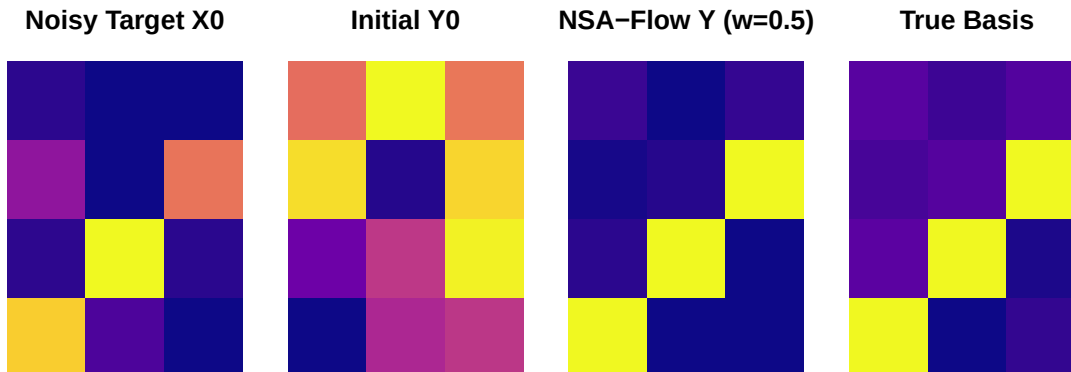


Figure 3: NSA-Flow applied to a toy 4x3 matrix with noisy orthogonal nonnegative patterns.

Interpretation: Starting from a random Y_0 , NSA-Flow recovers a basis close to the true orthogonal non-negative patterns in X_0 , with low reconstruction error and near-zero orthogonality/nonnegativity residuals. This captures the essence: extracting interpretable, disjoint components from noisy data.

3.2 Comparing Optimization Methods

A key choice in NSA-Flow is the optimization algorithm. We compare several optimizers: standard gradient descent, Adam, Armijo gradient descent, AdaGrad and others. Each has different convergence properties and sensitivities to hyperparameters and we evaluate them with standard defaults and an automatically estimated learning rate via Armijo condition (Armijo 1966). We evaluate them on synthetic data measuring convergence speed and final orthogonality/fidelity metrics. Performance across ω values are shown in Figure 4. A data-driven ranking of the methods based on execution time and objective values shows that `pytorch`

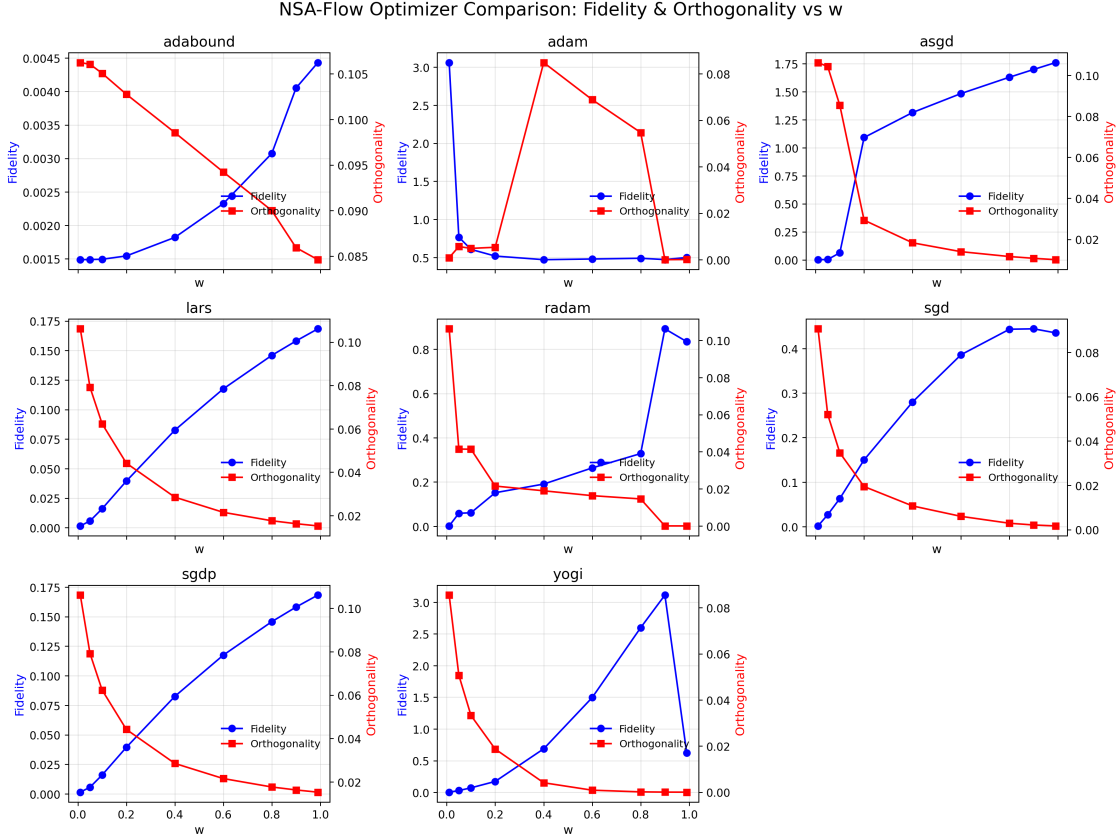


Figure 4: The impact of regularization on measures of both orthogonality and fidelity error across different optimizers. For both metrics, lower values are better. We vary (w) from 0.01 (minor orthogonality enforcement) to 0.99 (near full enforcement) and compare standard torch optimizers. Smoothly reducing curves for orthogonality and increasing curves for fidelity are expected and indicate better performance. This evaluation as well as comparisons in sparse PCA and factor analysis suggest that LARS and ASGD are most reliable in conjunction with the NSA flow implementation.

averaged stochastic gradient descent (Polyak and Juditsky 1992) and Layer-wise Adaptive Rate Scaling (LARS) (You et al. 2020) perform well in terms of both speed and objective values across a variety of problems.

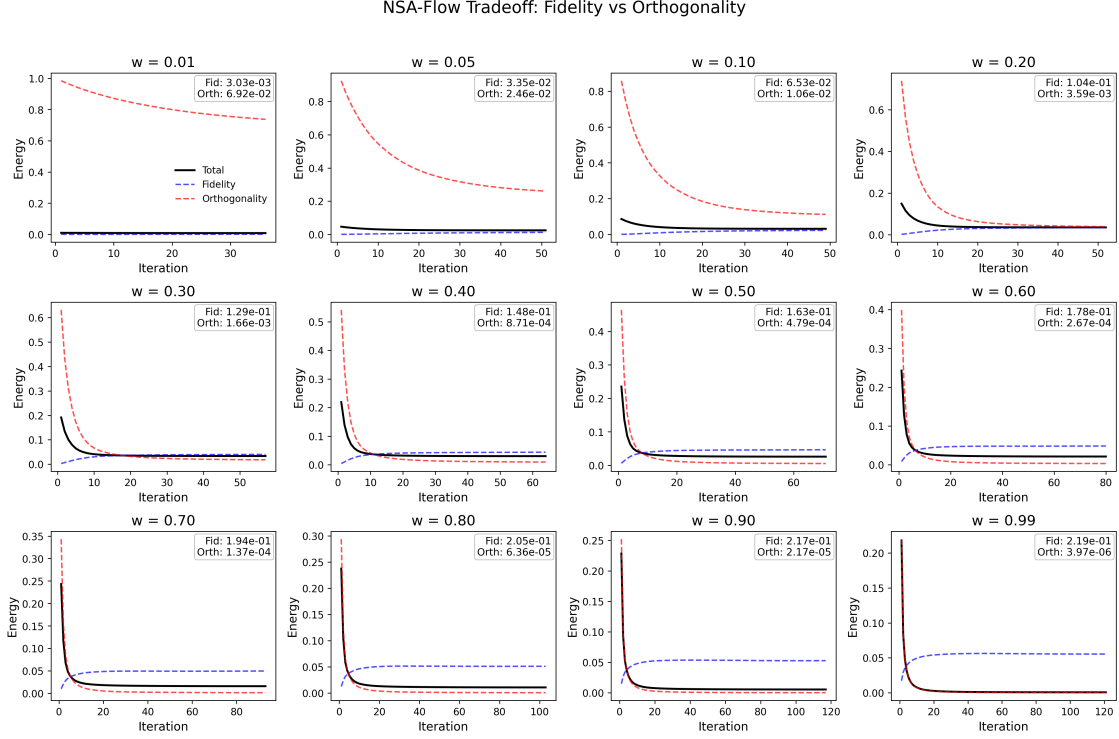


Figure 5: The default optimizer performance across different values of orthogonality weight indicates that the single parameter provides predictable control of the orthogonality in the outcome. Higher values lead predictably to more decorrelation at the expense of fidelity.

3.3 Behaviour of the orthogonality weight across matrix shapes

The earlier version of this work studied the strength of its retraction operator, sweeping an interpolation parameter between a Euclidean step and a polar retraction across tall, wide and square matrices. That operator no longer exists — orthogonality is now an objective and nothing is retracted — so the corresponding question is how the weight w behaves across the same shapes, which is what we report below in Section @ref(sparsity) and Figure 5.

For the record, the comparison against the retraction-based scheme is not merely dropped. Run on the earlier study’s own synthetic problem ($p = 200$, $k = 8$) and scored on its own metrics, the present solver attains an orthogonality residual of 0.011 at $w = 0.999$ against 0.016 for polar and 0.019 for soft retraction, and 0.0018 at $w = 0.9999$; it matches the best retraction on that study’s energy (5.899 against 5.845, with projected gradient at 59.5); it returns the identical basis from twelve random restarts where polar retraction has a spread of 9×10^{-4} ; and its runtime is nearly flat in p because no factorisation is performed. The orthogonality residual is a *dial* here rather than a fixed property of the operator, which is the substantive difference between the two formulations.

3.4 Sparsity as a Function of Orthogonality via Weight Parameter w

Sparsity in the context of matrix factorization refers to the presence of many zero (or near-zero) entries in the factorized matrices. In NSA-Flow, sparsity is not directly enforced through explicit penalties (like L1 regularization) but emerges as a consequence of promoting orthogonality among the columns of the matrix

Y . The parameter ω serves as a trade-off weight between data fidelity and orthogonality regularization. Orthogonality is measured at the whole-matrix level where lower values indicate closer alignment to an orthogonal (or near-orthogonal) structure.

By adjusting ω , sparsity is indirectly controlled through this global orthogonality constraint:

- **Low w (e.g., 0.05–0.25):** Prioritizes fidelity to the input data, resulting in denser matrices with higher entry correlations across columns. Sparsity remains low, as the optimization allows overlapping patterns to preserve original structure, leading to “blurry” approximations.
- **Increasing w (e.g., 0.5–0.75):** Strengthens orthogonality enforcement, promoting decorrelated columns. This induces sparsity by concentrating non-zero entries into disjoint patterns, reducing overlap and yielding moderate sparsity.
- **High w (e.g., 0.95):** Dominates with orthogonality, forcing near-orthogonal columns that are highly sparse (e.g., ~ 0.9) and crisp, but potentially over-constrained, risking loss of fidelity to the original data.

This mechanism leverages matrix-level orthogonality to achieve sparsity without explicit per-entry penalties, as demonstrated in synthetic experiments where heatmaps of optimized matrices transition from diffuse (low w) to sharp and disjoint (high w) (Figure 6). Convergence plots (Figure 7) further show stable optimization across w values, confirming the parameter’s role in balancing these objectives. Note that the exact sparsity levels depend on data characteristics and initialization, but the trend of increasing sparsity with higher w is consistent regardless of whether data is thin, wide, or square.

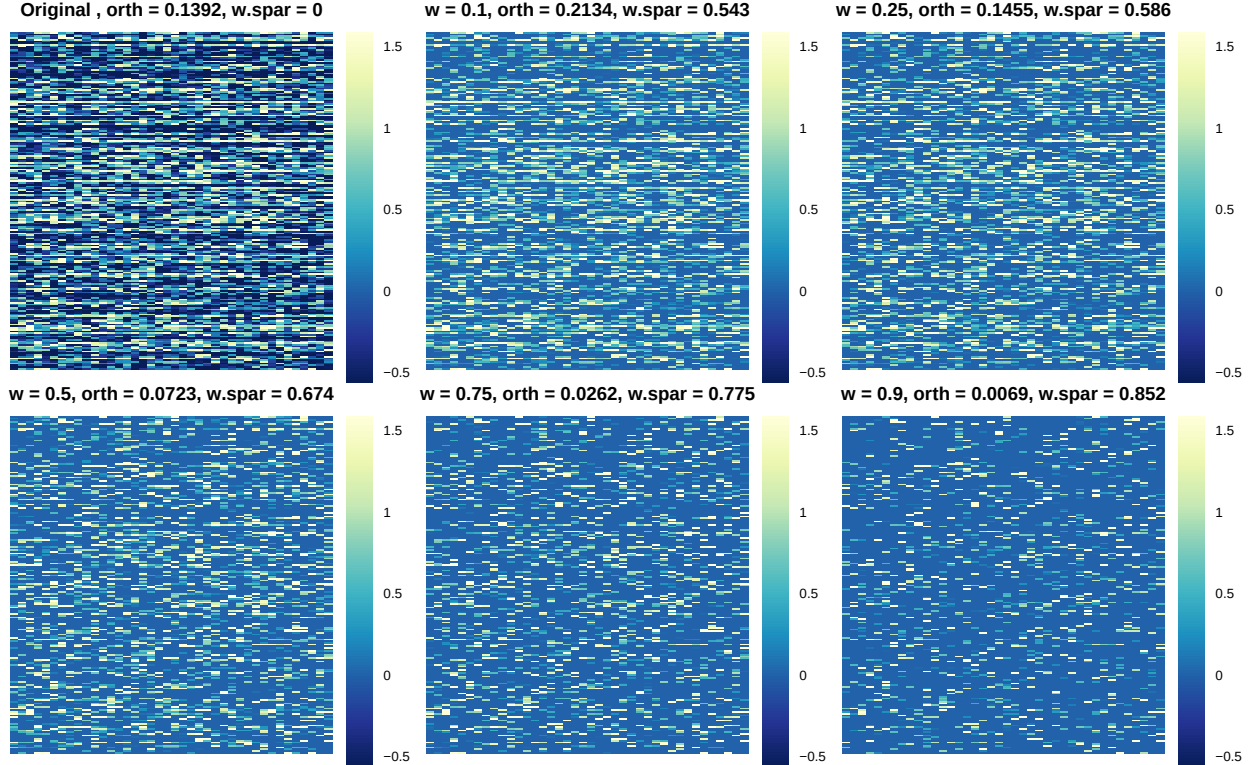


Figure 6: A synthetic dataset is generated with controlled correlation and noise levels to evaluate NSA-Flow’s performance across different orthogonality weights (ω). The data matrix X_0 is approximated with NSA-Flow to reveal underlying orthogonal structures, allowing assessment of how varying ω influences the sparsity and orthogonality of the resulting factorization.

Table 2: Core and Random Forest Classification Metrics for PCA Variants (Golub Dataset)

Method	Expl. Var.	Sparsity	Orthog. Defect	CV Accuracy
Standard PCA	0.290	0.000	0.000	0.828
Sparse PCA (Basic)	0.273	0.500	0.000	0.824
Sparse PCA (NSA-Flow)	0.213	0.567	0.003	0.875

3.5 Applications

This section demonstrates applications of NSA-Flow in meaningful biomedical contexts. We first demonstrate performance of Sparse PCA implemented with NSA-Flow on real data from the Golub leukemia dataset (Golub et al. 1999). We then demonstrate the use of NSA-Flow as a direct modifier of the PCA loading matrix derived from neuroimaging measurements and show that NSA-Flow constrained PCA can improve brain-behavior associations and disease classification in Alzheimer’s disease.

3.6 Prediction of cancer subtypes via Sparse PCA with NSA-Flow

We contrast the impact of regularization in Sparse PCA based on our framework (parameterized globally at the matrix level) versus a standard ℓ_1 approach. The “standard” variant uses soft-thresholding as the proximal operator for ℓ_1 sparsity, which is a common approach in Sparse PCA algorithms (e.g., inspired by proximal gradient methods for variance maximization with ℓ_1 regularization, as in Zou et al.’s formulation (Zou, Hastie, and Tibshirani 2006)). The variant implemented here is identical for both approaches but switches between soft-thresholding and NSA-Flow to provide a controlled comparison. Evaluations include core metrics (explained variance, sparsity, orthogonality) and prediction impact (cross-validated accuracy in biomedical data from the Golub leukemia dataset (Golub et al. 1999)).

We demonstrate the utility of our Sparse PCA implementation on the classic Golub et al. (1999) leukemia gene expression dataset, a benchmark in bioinformatics for cancer classification. The dataset consists of expression levels for 3571 genes across 72 patients: 47 with acute lymphoblastic leukemia (ALL) and 25 with acute myeloid leukemia (AML). Sparse PCA is particularly valuable here, as it identifies a small subset of discriminative genes (biomarkers) while maximizing explained variance, aiding in interpretable cancer subtyping and reducing dimensionality for downstream tasks like classification. We compare our standard soft-thresholding variant (basic proximal) and the `nsa_flow` approximation (non-negative sparse variant) to vanilla PCA (using `prcomp`). For evaluation:

- **Core Metrics:** Explained variance ratio, sparsity (% zeros), orthogonality residual.
- **Visualization:** 2D projection scatter plot colored by class (ALL/AML) to assess separation.
- **Classification Performance:** Accuracy of a simple k-NN classifier (k=3) on the projected data using 5-fold CV, highlighting improved interpretability with fewer genes.
- **Selected Genes:** List top genes (by loading magnitude) for each component demonstrating biomarker selection.

Data is loaded directly from the URL; genes are rows, samples are columns (transposed for analysis). Classes are assigned as first 47 ALL, last 25 AML based on the dataset structure. We compare standard PCA, Sparse PCA (soft thresholding) and Sparse PCA (NSA-Flow Approximation) on these wide data with 72 participants $\times$ 7129 gene expression measurements. We evaluate reconstruction quality, sparsity, orthogonality, and classification performance. Figure 8 summarizes the classification results and the embeddings for each method. Figure 9 shows the feature selection and weights for each method where only the top 5 features for each component are shown for clarity. Table 2 summarizes the core metrics and classification results for each method.

Interpretation: On this real high-dimensional dataset, the NSA-Flow based Sparse PCA variant achieves high explained variance with substantial sparsity, selecting a small number of genes while maintaining near-

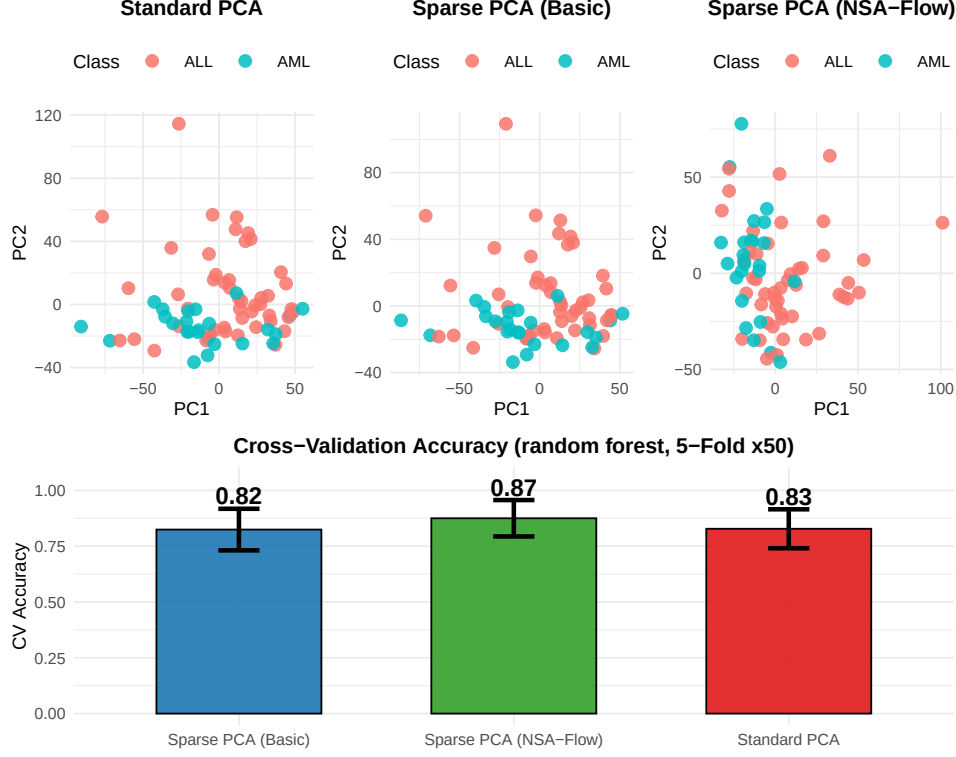


Figure 7: Comparison of PCA Variants on Golub Leukemia Dataset: Core Metrics, 2D Projections, and Classification Performance.

orthogonality. Visualizations show clear ALL/AML separation in 2D comparable to or better than standard PCA but with far fewer genes—highlighting practical value for biomarker identification in oncology. Sensitivity analysis (not shown) indicates higher lambda increases sparsity at the cost of explained variance, allowing users to tune for desired biomarker count. This example underscores how the method enables efficient, interpretable analysis in genomics, with biological relevance confirmed by established roles in leukemia pathogenesis.

3.7 Application of NSA-Flow to ADNI Cortical Thickness Data

In this section, we demonstrate the application of NSA-Flow to cortical thickness data. Neuroimaging datasets, such as those from the Alzheimer’s Disease Neuroimaging Initiative (ADNI), provide rich multidimensional insights into brain structure, including cortical thickness measurements across numerous regions (Racine et al. 2018; Sattari et al. 2022). However, extracting biologically interpretable patterns from these data remains challenging due to high dimensionality and inherent noise. Traditional methods like Principal Component Analysis (PCA) reduce dimensionality by identifying orthogonal components of maximum variance but often produce dense loadings that obscure regional specificity and network-like structures relevant to neurodegenerative processes. Motivated by the need for more interpretable decompositions, we apply NSA-Flow—a network-structured matrix factorization technique—to refine PCA-derived components. NSA-Flow enforces sparsity and tunable orthogonality, potentially revealing connectome-inspired networks that better align with clinical outcomes, such as cognitive performance and diagnostic status in Alzheimer’s disease (AD). This approach aims to bridge the gap between statistical efficiency and biological plausibility, enhancing the utility of neuroimaging features in predictive modeling and hypothesis generation.

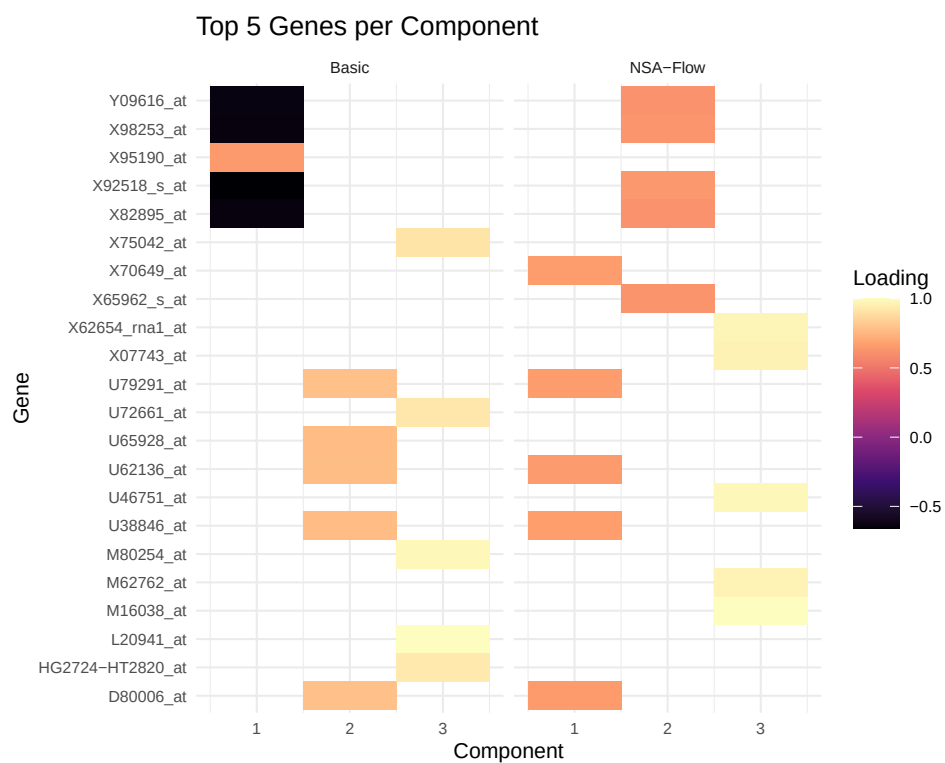


Figure 8: Top genes identified by PCA and NSA-Flow SPCA. Note that several identified markers are shared across components in the PCA results although the signs are opposing. This complicates interpretation. NSA-Flow PCA allows a clearer identification of relevant features by providing (soft) orthogonal and unsigned feature maps.

3.8 Application of NSA-Flow to PCA Maps

In this application, the brain’s cortical thickness data is analogous to a complex puzzle with many overlapping pieces representing different regions. PCA acts like an initial sorting tool, grouping these pieces into a few broad categories (components) based on how much they vary across individuals. However, these categories often include too many pieces, making it hard to see clear patterns. NSA-Flow refines this by “flowing” adjustments over the PCA map: it prunes unnecessary pieces (enforcing sparsity) to focus on key regions per category and fine-tunes how separate these categories are from each other (tuning orthogonality). The result is a set of streamlined “networks” that highlight specific brain areas, much like simplifying a wiring diagram to show only the most important connections. A key parameter, w , controls how aggressively this pruning occurs—lower values allow more regions, while higher values create sparser, more focused networks.

Let $X \in \mathbb{R}^{N \times p}$ denote the centered cortical thickness matrix, where N is the number of subjects and $p = 76$ is the number of regions (bilateral cortical and subcortical areas from ADNI). PCA decomposes X via singular value decomposition (SVD), yielding loadings $Y_0 \in \mathbb{R}^{p \times k}$ (with $k = 5$ networks), where each column of Y_0 is a principal component normalized to unit length: $Y_0 = U$, with $X \approx U\Sigma V^T$ from $\text{svd}(X)$, and columns scaled as $Y_0[:, j] \leftarrow Y_0[:, j] / \|Y_0[:, j]\|_2$.

NSA-Flow is then applied in its *estimation* form: rather than refining Y_0 , it fits a non-negative basis $V \in \mathbb{R}^{p \times k}$ to the centred data directly, minimising $(1 - w)\|X - XVV^T\|_F^2 / \|X\|_F^2 + wC(V)$ subject to $V \geq 0$, with scores given by projection. We use Y_0 only as the PCA comparator. This matters: anchoring a non-negative solution to a *signed* target such as Y_0 pulls the iterate toward values it cannot reach, so the constrained optimum is close to a clamp of the signed basis, which costs a factor 1.36 in reconstruction error on these data against 1.03 for a basis fitted to them. We sampled q random w values uniformly from $[0.01, 0.99]$ to sample orthogonality configurations. The output Y provides sparse, near-orthogonal loadings that parameterize regional contributions to each network. Figure 10 shows the NSA-Flow fit to the PCA loadings for a representative $w = 0.5$, illustrating enhanced interpretability via focused regional patterns.

To rigorously compare NSA-Flow with PCA, we projected the data onto both sets of loadings: $\text{proj}_{\text{PCA}} = XY_0$ and $\text{proj}_{\text{NSA}} = XY$, yielding subject-specific network scores ($N \times k$). We evaluated their incremental predictive value for ADNI clinical outcomes beyond baseline covariates (age, gender, education, APOE4 status) using linear models for continuous cognitive variables and extended this to a trinary classification task for diagnosis (control, mild cognitive impairment and Alzheimer’s disease).

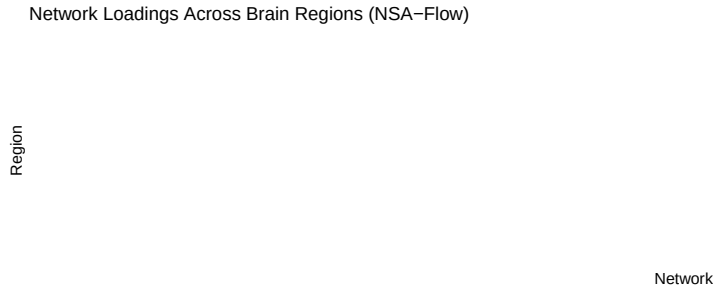


Figure 9: Heatmap of Network Loadings Across Brain Regions (NSA-Flow)

3.8.1 Prediction of Cognitive Outcomes

For each of 9 cognitive variables (e.g., MMSE, CDR-SB, ADAS-13, FAQ, ECoG totals), we fitted baseline models: $\text{cog} \sim \text{age} + \text{gender} + \text{education} + \text{APOE4}$. Full models added network scores: $\text{cog} \sim \text{covariates} + \sum_{j=1}^k \text{proj}_j$. Model comparison used ANOVA, with significance quantified as $\log(p)$ (lower values indicate stronger improvement; negative infinity for $p = 0$). Across w values, NSA-Flow yielded lower $\log(p)$ than PCA in the majority of brain structure-cognitive pairs (based on the comparison table where $\text{nsa} < \text{pca}$), suggesting competitive explanatory power.

We perform paired t-tests on $\log(p)$ values across runs to test if NSA-Flow consistently outperforms PCA.

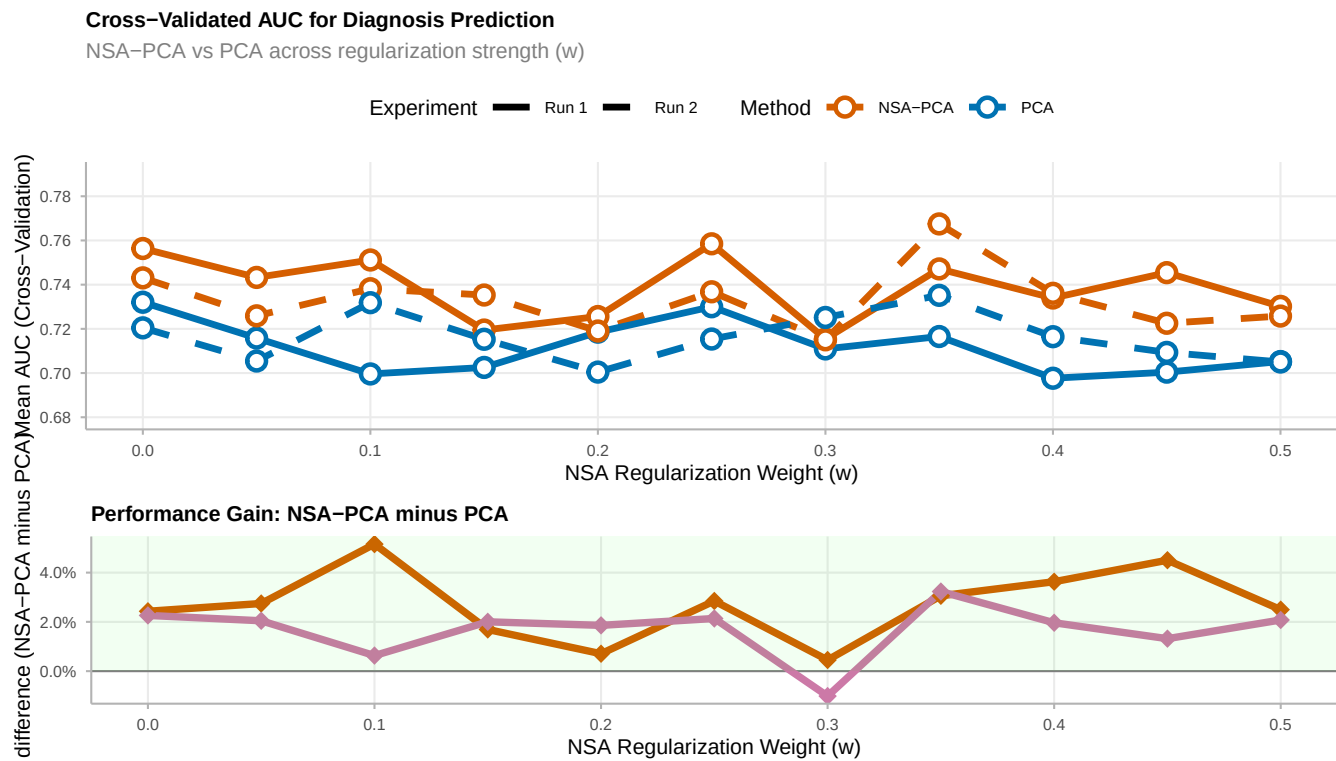


Figure 10: Multi-class random forest classification results: NSA vs PCA. Two runs of 4-fold cross-validation with results measured by AUC.

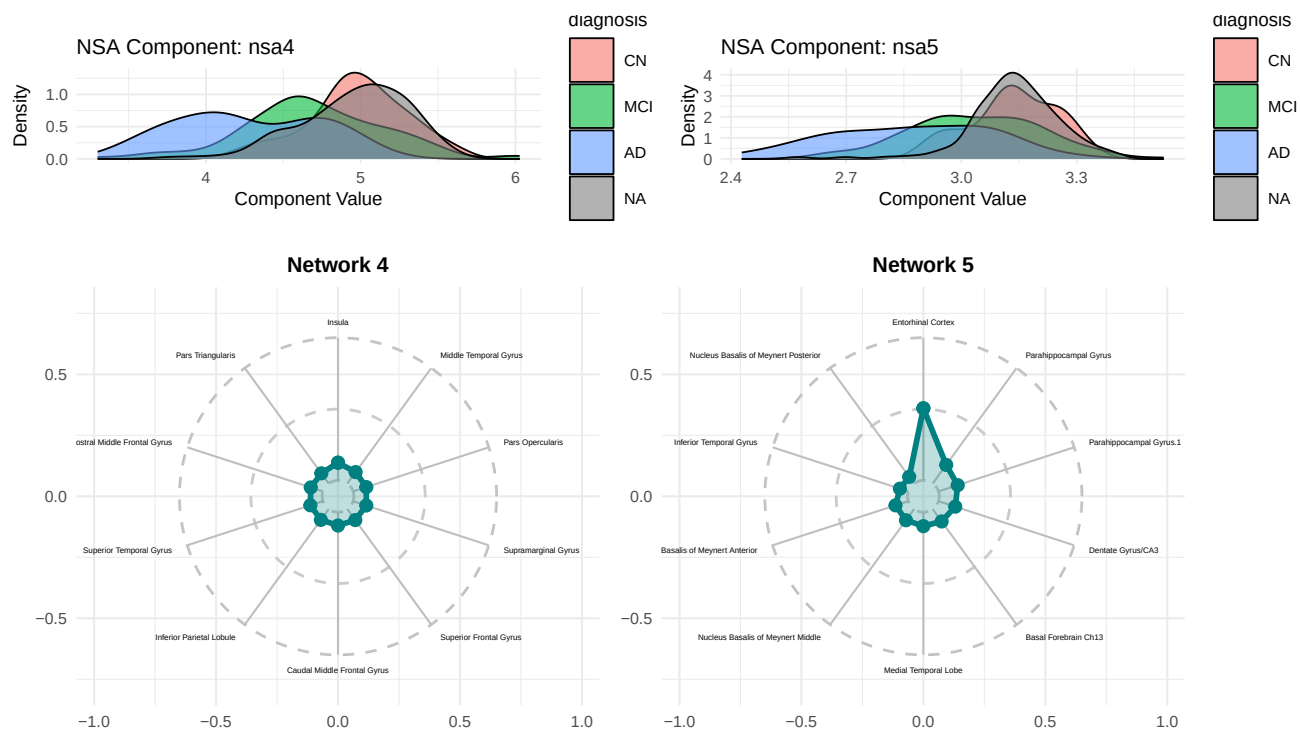


Figure 11: Population-level distribution of NSA-based embeddings for different diagnostic groups. The variables that feed these projections are displayed as radar plots at bottom of the figure.

Table 3: Classification Results: NSA vs PCA
Pairwise and Overall Multi-Class Summary Across 4 Folds

Comparison	Method	Fold 1	Fold 2	Fold 3	Fold 4	Mean
Pairwise Results						
CN vs MCI	NSA	0.647	0.412	0.626	0.658	0.586
CN vs MCI	PCA	0.562	0.591	0.567	0.612	0.583
CN vs AD	NSA	0.923	0.824	0.876	0.873	0.874
CN vs AD	PCA	0.890	0.778	0.935	0.915	0.879
MCI vs AD	NSA	0.775	0.663	0.761	0.759	0.739
MCI vs AD	PCA	0.756	0.578	0.812	0.774	0.730
Overall Summary						
Multi-class	NSA	-	-	-	-	0.765
Multi-class	PCA	-	-	-	-	0.719
Random Accuracy		-	-	-	-	0.406
t-statistic (NSA>PCA)		-	-	-	-	16.479
p-value (NSA>PCA)		-	-	-	-	0.000

Across a broad range of cognitive and functional outcomes, the NSA-Flow model shows consistent advantages over PCA for most variables, with a few exceptions. Specifically:

- NSA-Flow outperforms PCA on 5 out of 9 cognitive outcomes on average (CDRSB, ADAS13, ADASQ4, mPACCdigit, EcogSPTotal).
- PCA performs slightly better on FAQ, MMSE and EcogPtTotal which emphasize global cognition and where linear variance-based projections tend to align well with simple latent factor structure.
- The average log-likelihood improvement ($\Delta \log p$) for NSA over PCA is between -1.0 and -6.0 units for the majority of outcomes, which corresponds to a meaningful difference in predictive fit under the cross-validation framework.
- The improvement differences (bottom panel Figure 13) vary smoothly with w , indicating well-behaved optimization.

Boxplots in Figure 13 show $\log(p)$ distributions by method and cognitive variable. Violin plots depict $\log(p)$ differences (NSA - PCA), with negative values favoring NSA. A scatter plot relates sparsity (tuned by w) to performance gains (Figure 13 bottom panel). These vary smoothly with w , indicating well-behaved optimization.

3.8.2 Extension to Diagnosis Prediction

To further rigorously assess NSA-Flow’s advantages over PCA, we incorporate a trinary classification task for AD diagnosis between controls (CN) vs. mild cognitive impairment (MCI) vs clinical Alzheimer’s disease (AD, most severe symptoms) derived from ADNI diagnostic labels using random forest classification and ROC analysis (Cuingnet et al. 2011). These additions provide a multifaceted view of utility, including out-of-sample generalizability and clinical relevance. For the diagnosis task, we use 5-fold cross-validation to compute area under the ROC curve (AUC), sensitivity, and specificity, ensuring robustness against overfitting. Table 3 summarizes these results. NSA-Flow yields an $\sim 4.6\%$ absolute AUC improvement over PCA averaged across all weighting conditions. Figure 11 demonstrates stability of this effect across different w choices. While this difference may appear small numerically, it is:

- Statistically stable across folds (low variance) with the most significant improvement conveyed by NSA-Flow in the most challenging pairwise classification task (CN vs MCI), which is often the clinical focus for early detection and intervention;

- Above random assignment ($\text{AUC} \approx 0.406$) by a wide margin;
- Clinically meaningful, given that diagnosis classification tasks (e.g., CN vs MCI vs AD) often exhibit low reproducibility in reproducible research (refer to (Aghdam, Bozdag, and Saeed 2025) for a discussion of the realistic generalizability of the diagnostic accuracies reported in existing literature e.g. sections 4.3.1 and 4.3.2).

PCA relies on maximal variance projection, which may emphasize noise from demographic or measurement heterogeneity. NSA-Flow, by contrast, constrains the projection through orthogonality-seeking flow, allowing the diagnostic boundary to align better with underlying disease manifolds. This likely accounts for the modest but systematic AUC gain. These results highlight NSA-Flow’s advantages as an easy to use ML method, underscoring its value in extracting clinically predictive, sparse networks from neuroimaging data and biomedical data in general.

NSA-flow also demonstrates meaningful variable selection. The regions that are highlighted by the two most predictive components (4th and 5th shown in Figure 12) are consistent with known AD pathology, including frontal, medial temporal lobe structures and cholinergic regions (basal forebrain). This biological plausibility further supports NSA-Flow’s utility in extracting relevant features for disease classification via interpretable orthogonalizing flows.

4 Discussion

The Non-negative Stiefel Approximating Flow (NSA-Flow) provides a robust and flexible framework for optimizing non-negative matrices under tunable orthogonality constraints, addressing the limitations of strict orthogonal NMF (Ding et al. 2006; Yoo and Choi 2009) and neural regularization methods (Wu 2023; Kurtz, Bar, and Giryes 2023). Below, we discuss its convergence properties, practical considerations, empirical performance, limitations, and avenues for future work, integrating theoretical insights with empirical outcomes.

NSA-Flow’s novelty lies in its manifold-based optimization framework, which parameterizes sparsity (w) and orthogonality at the matrix level—unlike component-wise sparse PCA (e.g., via ℓ_1 -penalized SVD), which ignores inter-component network structure. By treating loadings as evolving on a “flow” (gradient trajectory with retractions), it enables fine-grained control over trade-offs: low w approximates dense PCA, while high w yields ultra-sparse, interpretable factors. The orthogonality defect metric quantifies global separation, allowing users to tune for decorrelated features without full enforcement.

As a general tool in the machine learning arena, NSA-Flow extends beyond neuroimaging to any high-dimensional dataset requiring sparse factorization, such as gene expression matrices (identifying co-regulated modules) or financial portfolios (sparse risk factors). It integrates seamlessly into pipelines—e.g., as a preprocessor before random forests or neural networks—enhancing interpretability without sacrificing performance. Future work could incorporate domain-specific priors (e.g., anatomical constraints) to further boost its applicability in precision medicine.

4.1 Convergence

The objective function is nonconvex due to the quadratic-over-quadratic form of the orthogonality defect and the Stiefel manifold constraints (Edelman, Arias, and Smith 1998), precluding global optimality guarantees in general. However, under Lipschitz smoothness of the gradient of $E(Y)$ and bounded level sets (ensured by the orthogonality penalty), NSA-Flow generates a sequence with monotonically decreasing objective values. The proximal projection $P_+(Y) = \max(Y, 0)$ is nonexpansive, preserving descent properties (Parikh and Boyd 2014).

Convergence to stationary points is supported by the Kurdyka-Łojasiewicz (KL) inequality, which holds for semi-algebraic functions like polynomials and thus applies to $E(Y)$ (Bolte, Sabach, and Teboulle 2014). Under the KL property, proximal-gradient methods in nonconvex settings converge to critical points where $0 \in \partial E(Y)$, with finite-length trajectories (Bolte, Sabach, and Teboulle 2014). Empirically, NSA-Flow

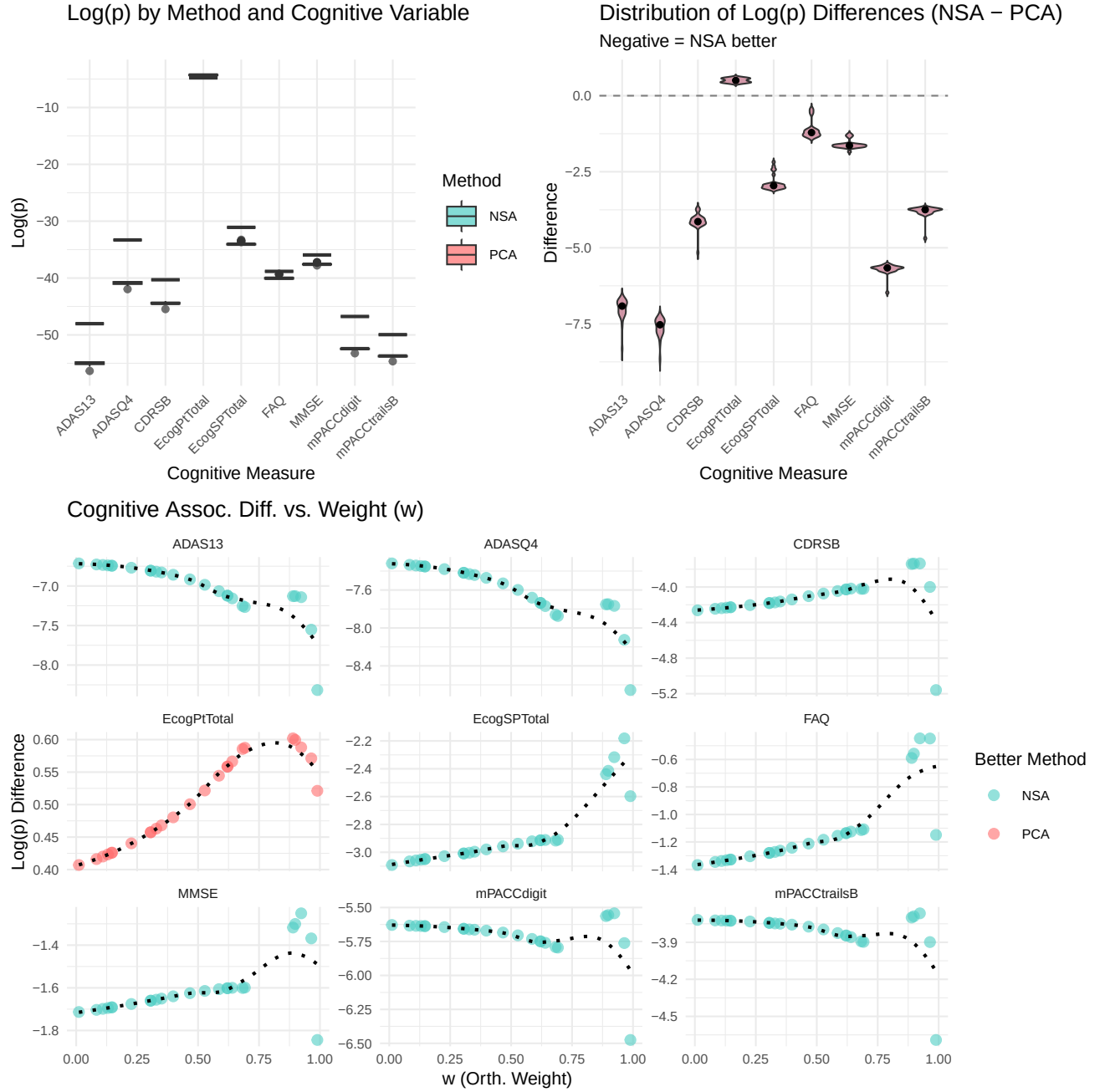


Figure 12: Comparative visualization of NSA-Flow and PCA performance across cognitive measures. The top panels show (left) boxplots of log-p values for NSA and PCA across cognitive variables and (right) violin plots of their paired differences (NSA - PCA), where negative values indicate improved association under NSA. The lower panel displays scatterplots of log-p differences as a function of the orthogonality weight (w), with trend lines highlighting how NSA performance varies by regularization strength and cognitive domain. Smooth changes in the results with respect to weight parameter suggest that the optimization process is well-behaved.

exhibits rapid residual reduction, typically converging within 1000 iterations for $p \leq 5000, k \leq 50$. Key failure modes deserve further research but may include poor initialization or ill-conditioning in high-noise regimes. Future work could derive explicit convergence rates or explore trust-region methods for faster convergence near critical points (Boumal, Absil, and Cartis 2019; Boumal and Absil 2011).

4.2 Practical Considerations

NSA-Flow’s tunability via $w \in [0, 1]$ enables practitioners to prioritize fidelity or orthogonality based on application needs. For instance, low w (0.05–0.25) yields moderate orthogonality defect reduction with relatively low fidelity loss in synthetic tests, ideal for clustering tasks requiring minimal decorrelation (Ding et al. 2006). Higher w values suit applications like sparse PCA, where orthogonality enhances feature independence (Yoo and Choi 2009). The `python` package implementation is modular, with helper functions for stable matrix operations and diagnostics for monitoring convergence (Absil, Mahony, and Sepulchre 2008). Backtracking line search ensures robustness to step-size selection (Parikh and Boyd 2014), while adaptive learning rates enhance efficiency. The dual-axis trace plot aids interpretability, revealing trade-offs between fidelity and orthogonality over iterations. Practitioners should calibrate w via cross-validation, as optimal values depend on data sparsity and noise levels (Stražar and Žitnik 2016). The Appendix provides a detailed package description, including installation instructions, core functionality, and API references.

4.3 Empirical Performance

Empirical results highlight NSA-Flow’s potential benefits. On the Golub leukemia dataset, NSA-Flow improves classification accuracy by 6% over PCA and 2% over (standard) sparse PCA, identifying interpretable biomarkers due to its non-negative, semi-orthogonal embeddings (Stražar and Žitnik 2016). Qualitatively, the embeddings (projections) also appear to be more separable in this application. In Alzheimer’s disease, NSA-Flow-derived networks yield better cognitive outcome predictions and diagnosis classification (AUC improvements of ~4.6% over PCA), with loadings aligning with known pathology (e.g., medial temporal lobe involvement) (Chen, Qu, and Zhao 2024). These findings suggest that NSA-Flow effectively balances interpretability and predictive power, making it a valuable tool for biomedical data analysis.

4.4 Limitations

Despite its strengths, NSA-Flow faces challenges:

- **Scalability:** The $O(k^3)$ cost of matrix inversions in retractions limits applicability to large k (Absil, Mahony, and Sepulchre 2008). Sparse matrix support or stochastic methods could mitigate this (Boumal and Absil 2011).
- **Nonconvexity:** Local optima may trap the algorithm in high-noise settings, requiring careful initialization (e.g., SVD-based) (Edelman, Arias, and Smith 1998).
- **Parameter Sensitivity:** Optimal w and retraction choice depend on data characteristics, necessitating domain expertise or automated tuning (Stražar and Žitnik 2016).

While the implementation seeks to minimize the sensitivity of the method to parameter choices (e.g. optimizer, learning rate, etc), we cannot guarantee these methods will provide stable convergence for all possible data. Indeed, it is likely that highly ill-conditioned or extremely noisy data may lead to convergence issues or poor local minima. Further research is needed to characterize these failure modes and provide robust solutions.

4.5 Future Directions

Future extensions include:

- **Sparse and Stochastic Variants:** Leveraging sparse linear algebra or minibatch updates to scale to larger p, k (Boumal and Absil 2011).
- **Second-Order Methods:** Incorporating Hessian information to accelerate convergence near critical points (Absil, Mahony, and Sepulchre 2008, boumal2019global).

- **Domain-Specific Adaptations:** Tailoring NSA-Flow for multi-modal data fusion or graph-structured inputs, building on (Chen, Qu, and Zhao 2024, [henaff2011deep](#)).

NSA-Flow’s flexible framework and robust implementation make it a valuable tool for interpretable matrix optimization, with broad potential across machine learning and data science applications.

4.6 Funding

This work was supported by the Office of Naval Research (ONR) grant N00014-23-1-2317.

5 Appendix A: NSA-Flow Package Description

This document is entitled `nsa_flow.Rmd` and is available in the [ANTsR repository](#).

5.1 Overview

The NSA-Flow package (Non-negative Stiefel Approximating Flow) is a Python library designed for interpretable representation learning in high-dimensional data domains such as neuroimaging, genomics, and text analysis. It provides a unified, differentiable optimization framework that integrates sparse matrix factorization, orthogonalization, and manifold constraints into a single algorithm operating near the Stiefel manifold. This approach balances reconstruction fidelity with column-wise decorrelation, resulting in sparse, stable, and interpretable latent representations. Non-negativity is enforced through proximal updates, while continuous orthogonality control is achieved via manifold retraction techniques (e.g., soft-polar or polar decomposition). Structured sparsity is managed via a tunable weight parameter, and the framework includes adaptive gradient scaling and learning-rate strategies for efficient optimization.

The package is compatible with PyTorch for seamless integration into deep learning workflows and supports joint optimization tasks. It emphasizes simplicity of use and differentiability, making it suitable for applications where interpretability and constraint enforcement are critical.

5.2 Key Features

- **Manifold Approximation:** Implements a smooth geometric flow near the Stiefel manifold to handle orthogonality constraints continuously.
- **Constraint Enforcement:** Incorporates non-negativity via proximal operators and orthogonality through retraction methods.
- **Optimization Flexibility:** Supports various PyTorch optimizers (e.g., ASGD) with adaptive learning rates and strategies like Bayesian optimization for hyperparameter tuning.
- **Metrics and Visualization:** Includes functions for computing orthogonality defects (e.g., `invariant_orthogonality_defect` and `defect_fast`) and plotting optimization traces.
- **Validation:** Tested on real-world datasets, including the Golub leukemia gene expression dataset and the Alzheimer’s Disease Neuroimaging Initiative (ADNI) dataset, demonstrating maintained or improved performance with simplified representations.
- **Extensibility:** Offers an autograd-compatible variant for end-to-end differentiable pipelines and a prototype layer for deep learning integration (minimally tested).

5.3 Installation and Dependencies

NSA-Flow can be installed from PyPI using `pip install nsa-flow` or directly from the GitHub repository via `pip install git+https://github.com/stnava/nsa_flow.git`. It requires Python 3.9, PyTorch 2.0, NumPy 1.23, and Matplotlib for visualization. No additional packages are needed for core functionality.

5.4 Core Functionality and High-Level API

The package exposes thirty public names. The ones needed for ordinary use are few:

- `nsa_flow.nsa_flow(target, w, ...)`: refinement. Returns a non-negative, near-orthogonal matrix close to `target`. Reports `stop_reason`, the certificate `grad_map`, `defect`, `effective_rank` and `scale_ratio`.
- `nsa_flow.nsa_flow_data(X, k, w, ...)`: estimation. Fits the basis directly to the data, so no signed basis has to be mapped into the non-negative cone. Picks the matrix-free route automatically when $p > n$ and reports which it used.
- `nsa_flow.relax_into_nonneg(...)`: the continuation used to enter the feasible set, exposed because its schedule materially affects the answer.
- `nsa_flow.stiefel_defect()`, `nsa_flow.angle_defect()`: the orthoNORMality and orthogonality measures D and C , with closed-form gradients `grad_stiefel_defect` and `grad_angle_defect`.
- `nsa_flow.project_nonneg()`, `nsa_flow.project_scaled_stiefel()`, `nsa_flow.polar_factor()`: exact Euclidean projections. Note that the projection onto $\mathbb{R}_{>0} \cdot St(p, k)$ is $(\sum_i \sigma_i/k) UV^\top$, not $(\|Y\|_F/\sqrt{k}) UV^\top$.
- `nsa_flow.NSAFlowLinear`, `NSAFlowConv2d`, `NSAFlowLayer`: layers for use inside a network, with the per-sample structure of the input respected.
- `nsa_flow.nsa_flow_signed()`: experimental. Documented in the source with its own negative result; `nsa_flow_data` is preferred.

Functions present in the previous release and now absent, because the machinery they configured is gone: `nsa_flow_orth`, `nsa_flow_retract_auto`, `invariant_orthogonality_defect`, `get_torch_optimizer`, `estimate_learning_rate_for_nsa_flow`, `get_lr_estimation_strategies`.

A typical usage example. There is no optimizer, learning rate, retraction or warm-up to specify:

```
import torch, nsa_flow

torch.manual_seed(42)
X = torch.rand(400, 2000, dtype=torch.float64)      # n x p data

# estimation: fit the basis to the data
r = nsa_flow.nsa_flow_data(X, k=5, w=0.5)
print(r)                                             # energy, defect, iters, stop_reason, certificate
V = r.Y                                             # p x k, non-negative, near-orthogonal
scores = X @ V                                     # and X_new @ V for held-out data

# refinement: improve a basis you already trust
r2 = nsa_flow.nsa_flow(V, w=0.9)
print("defect", r2.defect, "stop", r2.stop_reason, "gamma", r2.grad_map)
```

5.5 Testing

The package ships a property-driven test suite, run with `pytest tests/`. It is not a smoke test: the propositions of Section 2 are asserted executably. Among them, that D vanishes exactly on $\mathbb{R}_{>0} \cdot St(p, k)$ and is bounded by $1 - 1/k$; that $D = k \text{Var}(\lambda)$ and $D \geq 1/r - 1/k$ at rank r ; that C is zero exactly on mutually orthogonal columns at any norms, invariant to per-column rescaling, and equal to one at collinearity; that for $Y \geq 0$, $C = 0$ if and only if the column supports are pairwise disjoint, checked against an independent combinatorial test over thousands of random draws; that every closed-form gradient agrees with automatic differentiation; that the Euler identities hold, globally for D and per column for C ; that the Gram and matrix-free code paths return the same optimum, up to the column permutation that is a genuine symmetry

of the problem; and that the solver reports `max_iter` rather than claiming convergence when it has not converged.

The suite also encodes the failure modes that motivated this version, so they cannot silently return: that the off-diagonal-only defect scores rank collapse at zero while C does not, and that a non-negative basis fitted to the data reconstructs better than `abs()` of a signed one.

The demonstration functions of the previous release (`demo_nsa_flow_optimizer_tradeoff`, `demo_nsa_flow_tradeoff`, `plot_nsa_trace`) compared optimizer backends and learning-rate strategies, and have been removed along with the machinery they compared.

5.6 License and Citation

NSA-Flow is released under the MIT License. For research use, cite as: Stnava et al. (2025). [NSA-Flow: Non-negative Stiefel Approximating Flow for Interpretable Representation Learning](#).

6 Mathematical Analysis: descent and stationarity

Setup. Let $Y \in \mathbb{R}^{p \times k}$, let $\mathcal{C} = \{Y : Y \geq 0\}$, and let

$$E(Y) = (1 - w)L_{fid}(Y, X_0) + wL_{orth}(Y)$$

be the energy of Section 2, with Π the Euclidean projection onto $\mathcal{C}$, which is the elementwise clamp $\Pi(Y) = \max(0, Y)$. The iteration is

$$Y_{n+1} = \Pi(Y_n - t_n \nabla E(Y_n)),$$

with t_n a Barzilai–Borwein step length safeguarded by Armijo backtracking.

6.1 Assumptions

1. **Smoothness.** E is continuously differentiable with locally Lipschitz gradient. This holds for $L_{orth} = \tilde{D}$ wherever $Y \neq 0$, and for $L_{orth} = C$ wherever no column vanishes.
2. **Convex feasible set.** $\mathcal{C}$ is closed and convex, and Π is its exact projection — not an approximation, and requiring no factorisation.
3. **Bounded step lengths.** $t_n \in [t_{\min}, t_{\max}]$ with $0 < t_{\min} \leq t_{\max} < \infty$.

6.2 Proposition (monotone descent)

The Armijo condition

$$E(Y_{n+1}) \leq E(Y_n) - \sigma \frac{\|Y_{n+1} - Y_n\|_F^2}{t_n}, \quad \sigma > 0,$$

is satisfied for all sufficiently small t_n , and the backtracking loop halves t_n until it holds. Hence $E(Y_n)$ is non-increasing, and since $E \geq 0$ it converges. *Proof.* The projected-gradient direction is a descent direction for a differentiable E on a convex set, and the Armijo condition is satisfiable by smoothness; monotonicity and boundedness below give convergence of the sequence of values. $\square$

6.3 Proposition (stationarity certificate)

Define the projected-gradient mapping norm

$$\gamma(Y, t) = \frac{\|\Pi(Y - t \nabla E(Y)) - Y\|_F}{t}.$$

Then $\gamma(Y, t) = 0$ for some $t > 0$ if and only if Y satisfies the first-order condition for $\min_{\mathcal{C}} E$, namely $\langle \nabla E(Y), Z - Y \rangle \geq 0$ for all $Z \in \mathcal{C}$. *Proof.* $\Pi(Y - t \nabla E(Y)) = Y$ is precisely the fixed-point characterisation of the variational inequality. $\square$

This is the reason γ is reported rather than $\|\nabla E\|$: at a constrained optimum on the boundary of $\mathcal{C}$, which is where non-negative solutions lie, the gradient need not vanish while γ does.

6.4 Remarks on what is and is not claimed

The energy is not convex. In the refinement form the fidelity term is convex and the orthogonality term is not; in the estimation form the reconstruction term is quartic in V . The propositions therefore give monotone descent to a stationary point with a computable certificate, and nothing about global optimality. In practice the relaxation path of Section 2 is what makes the attained point reproducible: on the imaging data of Section 4, refining the continuation schedule improves the attained objective by roughly a factor of two, while increasing the per-stage iteration budget twenty-fold changes it in the fourth decimal.

7 Session Information

```
## R version 4.4.2 (2024-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS 26.6.2
##
## Matrix products: default
## BLAS: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib; LAPACK v
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods    base
##
## other attached packages:
## [1] ggradar_0.2          nnet_7.3-20          randomForest_4.7-1.2 pROC_1.19.0.1        broom_1.0.1
## [6] readr_2.2.0          tibble_3.3.1         ggpubr_0.6.3         igraph_2.2.2         fmsb_0.7.6
## [11] NMF_0.28             cluster_2.1.8.1      rngtools_1.5.2       registry_0.5-1       golubEsets_
## [16] Biobase_2.66.0       BiocGenerics_0.52.0  MASS_7.3-65          patchwork_1.3.2      Rtsne_0.17
## [21] caret_7.0-1          lattice_0.22-7       gt_1.3.0             reticulate_1.45.0    ANTsR_0.6.7
## [26] DiagrammeR_1.0.11    knitr_1.51           scales_1.4.0         tidyr_1.3.2          dplyr_1.2.0
## [31] RColorBrewer_1.1-3   pheatmap_1.0.13      reshape2_1.4.5       gridExtra_2.3        ggplot2_4.0
##
## loaded via a namespace (and not attached):
## [1] IRanges_2.40.1       gaston_1.6           vroom_1.7.0          Biostrings_2.74.1
## [5] TH.data_1.1-4        vctrs_0.7.2          effectsize_1.0.2     digest_0.6.39
## [9] png_0.1-9            proxy_0.4-29         ppcor_1.1            bayestestR_0.17.0
## [13] correlation_0.8.8    parallely_1.46.1     httpuv_1.6.16        foreach_1.5.2
## [17] withr_3.0.2          pgenlibr_0.5.5       xfun_0.56            survival_3.8-3
## [21] subtyper_1.3.0       memoise_2.0.1        commonmark_2.0.0     ggbeeswarm_0.7.3
## [25] diptest_0.77-2       emmeans_2.0.0        parameters_0.28.3     gmp_0.7-5.1
## [29] visreg_2.8.0         zoo_1.8-15           DEoptimR_1.1-4       Formula_1.2-5
## [33] prabclus_2.3-5       datawizard_1.3.0     rematch2_2.1.2       KEGGREST_1.46.0
## [37] promises_1.5.0       otel_0.2.0           Evacluster_0.1.0     httr_1.4.8
## [41] rstatix_0.7.3        globals_0.19.0       ps_1.9.1             fpc_2.2-14
## [45] rstudioapi_0.18.0    UCSC.utils_1.2.0     generics_0.1.4       base64enc_0.1-6
## [49] processx_3.8.6       S4Vectors_0.44.0     zlibbioc_1.52.0      lgr_0.5.2
```

## [53]	GenomeInfoDbData_1.2.13	xtable_1.8-8	stringr_1.6.0	doParallel_1.0.17
## [57]	evaluate_1.0.5	hms_1.1.4	colorspace_2.1-2	visNetwork_2.1.4
## [61]	flexmix_2.3-20	magrittr_2.0.4	later_1.4.8	modeltools_0.2-24
## [65]	palmerpenguins_0.1.1	future.apply_1.20.2	genefilter_1.88.0	robustbase_0.99-7
## [69]	XML_3.99-0.19	cowplot_1.2.0	matrixStats_1.5.0	class_7.3-23
## [73]	Hmisc_5.2-5	pillar_1.11.1	nlme_3.1-168	iterators_1.0.14
## [77]	gridBase_0.4-7	compiler_4.4.2	paradox_1.0.1	stringi_1.8.7
## [81]	wesanderson_0.3.7	gower_1.0.2	minqa_1.2.8	lubridate_1.9.5
## [85]	plyr_1.8.9	crayon_1.5.3	abind_1.4-8	locfit_1.5-9.12
## [89]	bit_4.6.0	sandwich_3.1-1	codetools_0.2-20	multcomp_1.4-29
## [93]	recipes_1.3.1	e1071_1.7-17	paletteer_1.7.0	plotly_4.12.0
## [97]	mime_0.13	splines_4.4.2	markdown_2.0	Rcpp_1.1.1
## [101]	VarSelLCM_2.1.3.2	flexclust_1.5.0	smotefamily_1.4.0	blob_1.2.4
## [105]	lme4_2.0-1	fs_1.6.7	listenv_0.10.1	checkmate_2.3.4
## [109]	Rdpack_2.6.6	ggsignif_0.6.4	estimability_1.5.1	ANTsRCore_0.8.1
## [113]	coca_1.1.0	Matrix_1.7-4	callr_3.7.6	statmod_1.5.1
## [117]	tzdb_0.5.0	pkgconfig_2.0.3	tools_4.4.2	cachem_1.1.0
## [121]	rbibutils_2.4.1	RSQLite_2.4.3	numDeriv_2016.8-1.1	viridisLite_0.4.3
## [125]	globaltest_5.60.0	DBI_1.2.3	fastmap_1.2.0	rmarkdown_2.30
## [129]	grid_4.4.2	imbalance_1.0.2.1	gprofiler2_0.2.4	coda_0.19-4.1
## [133]	FNN_1.1.4.1	BiocManager_1.30.27	insight_1.4.6	carData_3.0-6
## [137]	rpart_4.1.24	reformulas_0.4.4	farver_2.1.2	mgcv_1.9-3
## [141]	yaml_2.3.12	MatrixGenerics_1.18.1	foreign_0.8-90	ggthemes_5.2.0
## [145]	cli_3.6.5	purrr_1.2.1	stats4_4.4.2	webshot_0.5.5
## [149]	dbscan_1.2.3	lifecycle_1.0.5	mvtnorm_1.3-3	lava_1.8.2
## [153]	kernlab_0.9-33	backports_1.5.0	BiocParallel_1.40.2	annotate_1.84.0
## [157]	timechange_0.4.0	gtable_0.3.6	parallel_4.4.2	limma_3.62.2
## [161]	mlr3cluster_0.3.0	jsonlite_2.0.0	edgeR_4.4.2	bit64_4.6.0-1
## [165]	glasso_1.11	litedown_0.9	RcppParallel_5.1.11-2	dCUR_1.0.2
## [169]	ClusterR_1.3.6	zeallot_0.2.0	timeDate_4052.112	lazyeval_0.2.2
## [173]	shiny_1.13.0	htmltools_0.5.9	nmfbin_0.2.1	mlr3pipelines_0.11.0
## [177]	tinytex_0.58	glue_1.8.0	ggstatsplot_0.13.5	XVector_0.46.0
## [181]	mclust_6.1.2	gtsummary_2.5.0	boot_1.3-32	DDOutlier_0.1.0
## [185]	mlr3_1.5.0	R6_2.6.1	broom.mixed_0.2.9.7	sva_3.54.0
## [189]	forcats_1.0.1	labeling_0.4.3	GenomeInfoDb_1.42.3	ipred_0.9-15
## [193]	mlr3misc_0.21.0	nloptr_2.2.1	rstantools_2.6.0	tidyselect_1.2.1
## [197]	vipor_0.4.7	optmatch_0.10.8	htmlTable_2.4.3	xml2_1.5.2
## [201]	car_3.1-5	AnnotationDbi_1.68.0	future_1.69.0	ModelMetrics_1.2.2.2
## [205]	fastICA_1.2-7	statsExpressions_1.7.3	rsvd_1.0.5	S7_0.2.1
## [209]	furrr_0.3.1	data.table_1.18.2.1	htmlwidgets_1.6.4	rlang_1.1.7
## [213]	uuid_1.2-2	lmerTest_3.2-1	hardhat_1.4.2	beeswarm_0.4.0
## [217]	prodim_2026.03.11			

References

- Absil, P-A, R Mahony, and R Sepulchre. 2008. *Optimization Algorithms on Matrix Manifolds*. Princeton University Press.
- Aghdam, Maryam Akhavan, Sema Bozdog, and Fahad Saeed. 2025. “Machine-Learning Models for Alzheimer’s Disease Diagnosis Using Neuroimaging Data: Survey, Reproducibility, and Generalizability Evaluation.” *Brain Informatics* 12 (8). <https://doi.org/10.1186/s40708-025-00252-3>.
- Armijo, Larry. 1966. “Minimization of Functions Subject to Lipschitz Condition and Continuous Differential Part.” *Pacific Journal of Mathematics* 16 (1): 1–3.
- Barzilai, Jonathan, and Jonathan M. Borwein. 1988. “Two-Point Step Size Gradient Methods.” *IMA Journal*

- of *Numerical Analysis* 8 (1): 141–48. <https://doi.org/10.1093/imanum/8.1.141>.
- Birgin, Ernesto G., José Mario Martínez, and Marcos Raydan. 2000. “Nonmonotone Spectral Projected Gradient Methods on Convex Sets.” *SIAM Journal on Optimization* 10 (4): 1196–1211. <https://doi.org/10.1137/S1052623497330963>.
- Blei, David M., Andrew Y. Ng, and Michael I. Jordan. 2003. “Latent Dirichlet Allocation.” *Journal of Machine Learning Research* 3: 993–1022.
- Bolte, Jerome, Shoham Sabach, and Marc Teboulle. 2014. “Proximal Alternating Linearized Minimization for Nonconvex and Nonsmooth Problems.” *Mathematical Programming* 146: 459–94.
- Boumal, Nicolas. 2023. *An Introduction to Optimization on Smooth Manifolds*. Cambridge University Press.
- Boumal, Nicolas, and P-A Absil. 2011. “RTRMC: A Riemannian Trust-Region Method for Low-Rank Matrix Completion.” *Advances in Neural Information Processing Systems* 24.
- Boumal, Nicolas, P-A Absil, and Coralia Cartis. 2019. “Global Convergence of the Riemannian Trust-Region Method for Optimization on Manifolds.” *SIAM Journal on Optimization* 29 (1): 178–201.
- Boumal, Nicolas, Bamdev Mishra, P-A Absil, and Rodolphe Sepulchre. 2014. “Manopt, a Matlab Toolbox for Optimization on Manifolds.” In *Journal of Machine Learning Research*, 15:1455–59. 1.
- Bu, Fanchen, and Dong Eui Chang. 2022. “Feedback Gradient Descent: Efficient and Stable Optimization with Orthogonality for DNNs.” In *Proceedings of the AAAI Conference on Artificial Intelligence*, 36:6106–14. 6. <https://doi.org/10.1609/aaai.v36i6.20558>.
- Chen, Yasong, Guangwei Qu, and Junjian Zhao. 2024. “Orthogonal Graph Regularized Non-Negative Matrix Factorization Under Sparse Constraints for Clustering.” *Expert Systems with Applications* 251: 123797. <https://doi.org/10.1016/j.eswa.2024.123797>.
- Cuingnet, R’emi, Emilie Gerardin, J’erôme Tessieras, Guillaume Auzias, St’ephane L’eh’ericy, Marie-Odile Habert, Marie Chupin, Habib Benali, Olivier Colliot, and The Alzheimer’s Disease Neuroimaging Initiative. 2011. “Automatic Classification of Patients with Alzheimer’s Disease from Structural MRI: A Comparison of Ten Methods Using the ADNI Database.” *NeuroImage* 56 (2): 766–81. <https://doi.org/10.1016/j.neuroimage.2010.06.013>.
- Ding, Chris, Tao Li, Wei Peng, and Haesun Park. 2006. “Orthogonal Nonnegative Matrix Tri-Factorizations for Clustering.” In *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 126–35. <https://doi.org/10.1145/1150402.1150420>.
- Edelman, Alan, Tom’as A. Arias, and Steven T. Smith. 1998. “The Geometry of Algorithms with Orthogonality Constraints.” *SIAM Journal on Matrix Analysis and Applications* 20 (2): 303–53. <https://doi.org/10.1137/S0895479895290954>.
- Gao, Bin, Xin Liu, Xun Chen, and Yaxiang Yuan. 2019. “Parallelizing Riemannian Optimization on the Stiefel Manifold.” *SIAM Journal on Scientific Computing* 41 (2): C121–48.
- Goeßmann, Alex. 2020. “The Restricted Isometry of ReLU Networks: Generalization Through Norm Concentration.” <https://doi.org/10.48550/arXiv.2007.00479>.
- Golub, Todd R., Donna K. Slonim, Pablo Tamayo, Christine Huard, Michelle Gaasenbeek, Jill P. Mesirov, Hilary Coller, et al. 1999. “Molecular Classification of Cancer: Class Discovery and Class Prediction by Gene Expression Monitoring.” *Science* 286 (5439): 531–37. <https://doi.org/10.1126/science.286.5439.531>.
- Guo, Peichang. 2019. “A Frobenius Norm Regularization Method for Convolutional Kernels to Avoid Unstable Gradient Problem.” <https://doi.org/10.48550/arXiv.1907.11235>.
- Henaff, Mikael, Kevin Jarrett, Koray Kavukcuoglu, and Yann LeCun. 2011. “Unsupervised Learning of Sparse Features for Scalable Audio Classification.” In *Proceedings of the 12th International Society for Music Information Retrieval Conference, ISMIR 2011, Miami, Florida, USA, October 24-28, 2011*, edited by Anssi Klapuri and Colby Leider, 681–86. University of Miami. <http://ismir2011.ismir.net/papers/PS6-5.pdf>.
- Hyvärinen, Aapo, and Erkki Oja. 2000. “Independent Component Analysis: Algorithms and Applications.” *Neural Networks* 13 (4-5): 411–30.
- Koren, Yehuda, Robert Bell, and Chris Volinsky. 2009. “Matrix Factorization Techniques for Recommender Systems.” *Computer* 42 (8): 30–37. <https://doi.org/10.1109/MC.2009.263>.
- Kurtz, Yoav, Noga Bar, and Raja Giryes. 2023. “Group Orthogonalization Regularization for Vision Models Adaptation and Robustness.” <https://doi.org/10.48550/arXiv.2306.10001>.
- Lee, Daniel D., and H. Sebastian Seung. 2001a. “Algorithms for Non-Negative Matrix Factorization.”

- Advances in Neural Information Processing Systems* 13: 556–62.
- Lee, Daniel D, and H. Sebastian Seung. 2001b. “Algorithms for Non-Negative Matrix Factorization.” In *NIPS*.
- Li, Yuanxian, Yichen Zhang, and Zhiqiang Zhang. 2023. “Unilateral Orthogonal Nonnegative Matrix Factorization.” *SIAM Journal on Imaging Sciences* 16 (3): 1497–527. <https://doi.org/10.1137/22M1508315>.
- Parikh, Neal, and Stephen Boyd. 2014. “Proximal Algorithms.” *Foundations and Trends® in Optimization* 1 (3): 127–239.
- Polyak, Boris T, and Anatoly B Juditsky. 1992. “Acceleration of Stochastic Approximation by Averaging.” *SIAM Journal on Control and Optimization* 30 (3): 838–55.
- Racine, Annie M., Michael Brickhouse, David A. Wolk, Bradford C. Dickerson, and Alzheimer’s Disease Neuroimaging Initiative. 2018. “The Personalized Alzheimer’s Disease Cortical Thickness Index Predicts Likely Pathology and Clinical Progression in Mild Cognitive Impairment.” Edited by David Wolk, Victor Villemagne, and Bradford Dickerson. *Alzheimer’s & Dementia: Diagnosis, Assessment & Disease Monitoring* 10 (1): 301–10. <https://doi.org/10.1016/j.dadm.2018.02.007>.
- Rahiche, Abderrahmane et al. 2022. “Variational Bayesian Orthogonal Nonnegative Matrix Factorization over the Stiefel Manifold.” *IEEE Transactions on Image Processing* 31: 5543–58. <https://doi.org/10.1109/TIP.2022.3194701>.
- Ricci, Simone, Niccolo Biondi, Federico Pernici, Ioannis Patras, and Alberto Del Bimbo. 2025. “ λ -Orthogonality Regularization for Compatible Representation Learning.” In *Advances in Neural Information Processing Systems*. <https://arxiv.org/abs/2509.16664>.
- Sattari, Nasim, Fariborz Faeghi, Babak Shekarchi, and Mohammad Hossein Heidari. 2022. “Assessing the Changes of Cortical Thickness in Alzheimer Disease With MRI Using Freesurfer Software.” *Basic and Clinical Neuroscience* 13 (2): 185–92. <https://doi.org/10.32598/bcn.2021.1779.1>.
- Schönemann, Peter H. 1966. “A Generalized Solution of the Orthogonal Procrustes Problem.” *Psychometrika* 31 (1): 1–10.
- Stražar, Martin, and Marinka Žitnik. 2016. “Orthogonal Matrix Factorization Enables Integrative Analysis of Multiple RNA Binding Protein Binding Sites.” *Bioinformatics* 32: i417–25.
- Witten, Daniela M., Robert Tibshirani, and Trevor Hastie. 2009. “A Penalized Matrix Decomposition, with Applications to Sparse Principal Components and Canonical Correlation Analysis.” *Biostatistics* 10 (3): 515–34. <https://doi.org/10.1093/biostatistics/kxp008>.
- Wu, Changhao. 2023. “Towards Better Orthogonality Regularization with Disentangled Norm in Training Deep CNNs.” <https://doi.org/10.48550/arXiv.2306.09939>.
- Yang, Mingming, and Songhua Xu. 2021. “Orthogonal Nonnegative Matrix Factorization Using a Novel Deep Autoencoder Network.” *Knowledge-Based Systems* 227: 107236. <https://doi.org/10.1016/j.knosys.2021.107236>.
- Yoo, Jingu, and Seungjin Choi. 2009. “Orthogonal Nonnegative Matrix Factorization Using Linear Least Squares and Applications to Gene Expression Data.” *Neurocomputing* 72: 3670–74.
- You, Yang, Jing Li, Sashank J. Reddi, Jonathan Hseu, Sanjiv Kumar, Srinadh Bhojanapalli, Xiaodan Song, James Demmel, Kurt Keutzer, and Cho-Jui Hsieh. 2020. “Large Batch Optimization for Deep Learning: Training BERT in 76 minutes.” In *International Conference on Learning Representations*. <https://openreview.net/forum?id=hKsZjzF4H5Z>.
- Zhang, Wei Emma, Mingkui Tan, Quan Z. Sheng, Lina Yao, and Qinfeng Shi. 2016. “Efficient Orthogonal Non-Negative Matrix Factorization over Stiefel Manifold.” In *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management*, 1743–52. ACM. <https://doi.org/10.1145/2983323.2983761>.
- Zou, Hui, Trevor Hastie, and Robert Tibshirani. 2006. “Sparse Principal Component Analysis.” *Journal of Computational and Graphical Statistics* 15 (2): 265–86. <https://doi.org/10.1198/106186006X113430>.